Estimación de la temperatura de punto de rocío (TDEW) a partir de la temperatura mínima sobre PISCOt

Motivación, problema y metodología: regresión lineal local sobre anomalías con un diseño paralelo bucket-año

Piero Palacios Bernuy Estuardo Oliver Campos Diego Sánchez Salazar

Gianmarco Mejía

2026-07-18

Resumen La temperatura de punto de rocío (TDEW, en adelante TD) es una variable climática fundamental: determina la presión de vapor real del aire y, con ello, la formación de rocío, el riesgo de heladas y la duración del período de humectación foliar que controla la infección por patógenos fúngicos en los cultivos. Como la TD rara vez se mide de forma directa en redes meteorológicas extensas, debe estimarse a partir de variables más disponibles, típicamente la temperatura mínima diaria (TMIN). Este trabajo aborda dicha estimación sobre el conjunto de datos grillados PISCOt para el Perú, comparando sus versiones v1.1 y v1.2, lo que implica ajustar un modelo para del orden de 300 000 ubicaciones espaciales a lo largo de cerca de 40 años de registro diario. Proponemos un modelo de regresión lineal local ponderada sobre anomalías, analizado en el modelo PRAM (variante CREW) e implementado mediante una organización *bucket-año* que agrupa las entradas en archivos compactos y reduce el sobrecosto de entrada/salida (E/S). Finalmente, se ejecutó un protocolo experimental de escalamiento fuerte y débil que midió tiempo de ejecución, aceleración y eficiencia en CPU (barrido de 1 a 32 procesos) y, en GPU, sobre una única partición A100 MIG (`a100_3g.20gb`) caracterizada por su rendimiento y su *roofline*, de forma idéntica sobre PISCOt v1.1 y v1.2. Los resultados confirmaron el modelo teórico: en CPU la eficiencia decae como $O(1/p)$ por contención del bus de memoria (techo de aceleración $S(\infty) \approx 17$), mientras que en GPU el pipeline está limitado por ancho de banda y el entrenamiento del dominio completo de $\approx 300\,000$ puntos tomó ≈ 14.7 minutos en una sola GPU.

Palabras clave: *temperatura de punto de rocío, temperatura mínima, regresión lineal local, anomalías climáticas, PISCOt, PRAM, escalamiento HPC, enfermedades fúngicas.*

Contenido

1 Introducción 2
1.1 La importancia de la temperatura de punto de rocío 2

1.2 El conjunto de datos PISCOt . . . 3
1.3 El problema a resolver . . . 3
1.4 Contribución y estructura . . . 4
2 Metodología . . . 4
2.1 Datos y variables . . . 4
2.2 Modelo de regresión lineal local . . . 5
2.3 Modelo PRAM y notación de complejidad . . . 6
2.4 Pipeline completo . . . 6
2.5 Algoritmo dominante: entrenamiento de coeficientes . . . 7
2.6 Complejidad global . . . 8
2.7 Escalabilidad teórica e isoeficiencia . . . 9
2.8 Implementación y ejecución de los experimentos . . . 10
3 Resultados . . . 12
3.1 Modelos de referencia para las curvas experimentales . . . 13
3.2 Granularidad de oleadas: cuántos ajustes corren a la vez . . . 14
3.3 Resultados en CPU (Trabajo A) . . . 14
3.4 Resultados en GPU (Trabajo B) . . . 16
3.5 Comparación PISCOt v1.1 vs v1.2 (Trabajo C) . . . 18
3.6 Recomendación . . . 20
4 Conclusiones . . . 20
5 Contribución . . . 21
Referencias . . . 21
Apéndice A. Pseudocódigo PRAM y análisis de complejidad completos . . . 23
A.1 Propósito . . . 23
A.2 Modelo PRAM . . . 23
A.3 Símbolos . . . 24
A.4 Cómo leer las fórmulas de complejidad . . . 24
A.5 Algoritmo 1: pipeline completo de TDEW . . . 25
A.6 Algoritmo 2: construir climatología . . . 26
A.7 Algoritmo 3: construir el dataset de entrenamiento bucket-año . . . 27
A.8 Algoritmo 4: fragmentar climatología por bucket . . . 29
A.9 Algoritmo 4b: fragmentar TMIN futuro por bucket . . . 29
A.10 Algoritmo 5: entrenar coeficientes de anomalía . . . 30
A.11 Algoritmo 6: verificar completitud . . . 33
A.12 Algoritmo 7: reparar DOYs fallidos . . . 34
A.13 Algoritmo 8: parchar coeficientes . . . 34
A.14 Algoritmo 9: pronóstico opcional (bucketizado) . . . 35
A.15 Tabla resumen de complejidad . . . 36
A.16 Complejidad global . . . 37
A.17 Interpretación HPC . . . 38
A.18 Notas prácticas para esta máquina . . . 40
A.19 Metodología experimental . . . 41

1 Introducción

1.1 La importancia de la temperatura de punto de rocío

La temperatura de punto de rocío es la temperatura a la cual el aire debe enfriarse, a presión constante, para que el vapor de agua que contiene alcance la saturación y comience a conden-

sarse. Es, por tanto, una medida directa del contenido de humedad del aire y fija la presión de vapor real empleada en el cálculo del balance hídrico y de la evapotranspiración de referencia mediante la ecuación de Penman–Monteith (Allen et al., 1998). Su relevancia abarca desde la hidrología y la predicción de heladas hasta, de manera especialmente crítica, la fitopatología agrícola.

El vínculo con la sanidad vegetal es directo. La mayoría de los hongos y bacterias fitopatógenos requieren agua líquida libre sobre la superficie de la planta para germinar e infectar; esa agua proviene de la lluvia, la niebla y, sobre todo, del rocío, que se forma cuando la temperatura de la superficie foliar desciende por debajo del punto de rocío del aire circundante (Huber & Gillespie, 1992). La **duración del período de humectación foliar** —principal predictor del riesgo de infección— suele estimarse a partir de variables meteorológicas como la depresión del punto de rocío y la humedad relativa, precisamente porque medirla de forma directa es costoso (Sentelhas et al., 2008). En consecuencia, disponer de campos espacialmente continuos y fiables de TD es un insumo de primer orden para los sistemas de alerta temprana de enfermedades.

Este punto es particularmente importante en el contexto peruano y andino. La papa, cultivo estratégico para la seguridad alimentaria de la región, es severamente afectada por el tizón tardío (*Phytophthora infestans*), cuya tasa de infección y esporulación depende de manera estrecha de la temperatura y de la humedad: las noches frescas con rocío abundante seguidas de días húmedos constituyen las condiciones ideales para la epidemia (Narouei-Khandan et al., 2020). Los modelos mecanicistas de pronóstico de tizón se alimentan explícitamente de series de temperatura y humedad, de modo que la calidad de las estimaciones de TD condiciona directamente la utilidad de estas herramientas de gestión.

1.2 El conjunto de datos PISCOt

PISCOt es el producto grillado de temperatura del aire para el Perú desarrollado por el Servicio Nacional de Meteorología e Hidrología (SENAMHI) dentro de la familia PISCO (*Peruvian Interpolated data of SENAMHI's Climatological and hydrological Observations*). Su versión más reciente, PISCOt v1.2, ofrece grillas diarias de temperatura del aire (máxima y mínima) a una resolución espacial de 0.01° para todo el territorio nacional en el período 1981–2020 (Huerta et al., 2023). La construcción del producto sigue cuatro etapas: control de calidad, relleno de vacíos, homogeneización de las estaciones y, finalmente, interpolación espacial; la versión v1.2 mejora sobre la v1.1 mediante datos adicionales, una secuencia de cálculo revisada y un control de versiones más estricto (Huerta et al., 2023).

Para este proyecto, PISCOt aporta la variable predictora (TMIN) sobre una grilla densa y homogénea. La temperatura de punto de rocío, en cambio, tiene una disponibilidad limitada e inconsistente entre versiones: **PISCOt v1.1 incluye TD solo hasta 2016, mientras que PISCOt v1.2 no incluye TD en absoluto**. Esta cobertura parcial es precisamente lo que motiva estimar la TD a partir de la TMIN, de modo que se obtenga un campo de TD completo y homogéneo para todo el período y coherente entre ambas versiones del producto. Comparar v1.1 y v1.2 permite además evaluar cómo los cambios en el producto subyacente se propagan al modelo de TD estimado.

1.3 El problema a resolver

El objetivo es estimar la TD diaria para cada celda de la grilla PISCOt usando la TMIN como predictor principal. La escala es masiva: del orden de $N \approx 300\,000$ ubicaciones espaciales (identificadas por un `ID`) a lo largo de $Y \approx 40$ años de registro diario, lo que supone del orden de miles de millones de registros por variable climática.

El reto no es solo de volumen, sino también estadístico. La temperatura cruda —tanto TD como TMIN— posee un ciclo estacional fuerte y predecible: cada ubicación es más cálida en verano y más fría en invierno. Si se modelara la TD cruda frente a la TMIN cruda, gran parte de la relación aparente sería simplemente el reflejo de que ambas variables siguen las estaciones, y no el acoplamiento físico genuino entre humedad y temperatura que interesa capturar. Por ello se trabaja sobre **anomalías**, restando a cada observación el valor típico de su ubicación para ese día del año (su climatología), tal como se define en la Ecuación 1:

$$\begin{aligned} TD_{anom}(t) &= TD(t) - TD_{clim}(ID, doy), \\ TMIN_{anom}(t) &= TMIN(t) - TMIN_{clim}(ID, doy). \end{aligned} \tag{1}$$

Modelar la relación día a día por separado para cada par `(ID, doy)` genera del orden de $N \times D \approx 300\,000 \times 366 \approx 1.1 \times 10^8$ ajustes de regresión independientes. Cada ajuste es diminuto, pero su número total convierte el cómputo en un problema intensivo que exige un diseño paralelo cuidadoso y motiva tanto el análisis de complejidad como los experimentos de escalamiento descritos en la metodología.

1.4 Contribución y estructura

El trabajo propone (i) un modelo de regresión lineal local ponderada sobre anomalías para estimar TD a partir de TMIN; (ii) una organización **bucket-año** del flujo de datos que agrupa las entradas en archivos compactos; (iii) un análisis de complejidad en el modelo PRAM que predice el rendimiento en clústeres de cómputo de alto desempeño (HPC); y (iv) un protocolo experimental que mide el escalamiento fuerte y débil en CPU y GPU sobre las dos versiones de PISCOt. La Sección de Metodología detalla los datos, el modelo estadístico, el modelo PRAM con su pseudocódigo y análisis de complejidad, la implementación y el diseño experimental.

2 Metodología

2.1 Datos y variables

La unidad de análisis es la celda de grilla (`ID`). De PISCOt (Huerta et al., 2023) se toman las series diarias de TMIN (predictor) y TD (objetivo) para cada `ID` durante el período de entrenamiento. Todo el protocolo se ejecuta de forma idéntica sobre PISCOt v1.1 y v1.2, suministrando cada versión al mismo pipeline mediante parámetros de ruta de entrada y escribiendo a raíces de resultados separadas, de modo que no difiere ningún código entre ejecuciones.

Para cada `ID` y día del año `doy` se define la climatología como la media histórica de cada variable, y las anomalías como la desviación respecto de dicha climatología. La regresión se ajusta sobre las anomalías, eliminando así el ciclo estacional. La notación empleada en todo el documento se resume en la Tabla 1.

Tabla 1: Símbolos y parámetros del modelo.

Símbolo	Significado	Valor típico
N	Número de ubicaciones espaciales (`ID`s)	$\approx 300\,000$
Y	Años de entrenamiento	≈ 40
D	Días por año	≤ 366
B	Número de buckets de `ID`	p. ej. 1024
p	Procesadores / trabajadores	variable
h	Semiventana local de `doy`	11
F	Coeficientes de regresión (intercepto + predictores)	5
m	Muestras por regresión local	$O(Yh)$

2.2 Modelo de regresión lineal local

El núcleo estadístico es una **regresión lineal local ponderada**: un método de regresión no paramétrica que, en lugar de ajustar una sola recta global, ajusta un modelo lineal simple dentro de una vecindad local de cada punto, ponderando las observaciones según su cercanía mediante una función de kernel (Cleveland, 1979; Fan & Gijbels, 1996). El diseño de este trabajo se inspira directamente en Hwang et al. (2019), quienes incorporan características de vecinos más cercanos (*nearest-neighbor*) dentro de una regresión lineal local ponderada para mejorar el pronóstico subestacional de temperatura y precipitación en el oeste de Estados Unidos. Para estimar el modelo de la ubicación `i` en el día del año `d`, no se usan únicamente las observaciones del día `d` —demasiado escasas a lo largo de 40 años—, sino todas aquellas cuyo día del año cae dentro de una **ventana circular local** de semianchura `h` alrededor de `d`, de modo que el día 365 es vecino del día 1. Las observaciones más próximas a `d` reciben mayor influencia mediante un peso **tricube** que decae suavemente hasta cero en el borde de la ventana (Ecuación 2):

$$w(t) = \left(1 - \left|\frac{\delta(doy(t), d)}{h}\right|^3\right)^3, \qquad \text{para} \quad |\delta| \leq h, \tag{2}$$

donde δ es la distancia circular entre días del año. En todas las corridas se fijó $\boldsymbol{h = 11}$, por lo que cada ajuste puede utilizar una ventana circular de $2h + 1 = 23$ días del año, centrada en el `doy` objetivo. Se empleó el kernel **tricube** y se exigieron al menos 15 observaciones completas para aceptar un ajuste.

La variable respuesta fue $y_t = TD_{anom}(t)$. El vector de características ejecutado tanto en CPU como en GPU fue

$$x_t = [1, TMIN_{anom}(t), TD_{anom}(t-1), TD_{anom}(t-2), TMIN_{anom}(t-1)],$$

es decir, un intercepto y cuatro predictores: `TMIN_anom`, `TD_anom_lag1`, `TD_anom_lag2` y `TMIN_anom_lag1`. Los rezagos se construyeron por `ID` después de ordenar cronológicamente las observaciones. Sobre la vecindad ponderada se resolvió una regresión por mínimos cuadrados ponderados, obteniendo el vector de coeficientes β almacenado para cada `(ID, doy)`. Con $F = 5$ coeficientes y $m = O(Yh)$ muestras de vecindad por ajuste, cada ajuste resuelve el sistema de ecuaciones normales

$$(X^\top W X)\,\beta = X^\top W y. \tag{3}$$

2.3 Modelo PRAM y notación de complejidad

El análisis paralelo se realiza en el modelo **PRAM** (Máquina de Acceso Aleatorio Paralela), el marco idealizado estándar para razonar sobre algoritmos paralelos (Cormen et al., 2009; JáJá, 1992). Se usa la variante **CREW** (*Concurrent Read, Exclusive Write*), que se ajusta de forma natural a este pipeline: muchos procesadores pueden leer concurrentemente las mismas tablas de entrada (lectura concurrente), mientras que cada trabajador es dueño de un bucket / `ID` / día del año de salida disjunto, de modo que nunca colisionan dos escrituras (escritura exclusiva). Las reducciones (sumas, medias, conteos) combinan m valores en aproximadamente $\log m$ pasos mediante un árbol de sumas por pares. La anotación `pardo` marca los bucles cuyas iteraciones son independientes y, por tanto, paralelizables.

Cada algoritmo se describe con tres costos (Cormen et al., 2009; JáJá, 1992): el **trabajo** W (número total de operaciones, independiente del número de procesadores), el **tiempo paralelo** T_p con p procesadores, y el **span** T_∞ (camino crítico, el tiempo con procesadores ilimitados). La forma recurrente del tiempo paralelo es

$$T_p \approx \frac{W}{p_{\text{eff}}} + \text{sobrecosto (coordinación, E/S, dependencias)} \ .$$

El **diseño bucket-año** es la decisión arquitectónica central. Un *bucket* es un grupo determinista de `ID`s definido por `bucket = ID mod B`. En lugar de que cada uno de los ~300 000 `ID`s vuelva a abrir los mismos archivos mensuales históricos, los datos se agrupan una sola vez en B buckets y se compactan por año; después, cada trabajador lee los archivos compactos de su bucket una única vez, convirtiendo una tormenta de lecturas diminutas y repetidas en unas pocas lecturas secuenciales grandes. La principal consecuencia es que el paralelismo efectivo de las etapas de cómputo queda acotado por el número de buckets, $p_{\text{eff}} \leq B$.

2.4 Pipeline completo

El pipeline encadena nueve algoritmos; cada uno se ejecuta en secuencia, pero es internamente paralelo. El Algoritmo 1 da la vista general.

Algoritmo 1 — Pipeline completo de TDEW.

```
Algorithm TDEW-PIPELINE
Input:
  TD, TMIN monthly files for all IDs and dates; Grid of expected IDs
  Parameters: B buckets, h local DOY half-window, p processors
Output:
  climatology; bucket-year training dataset;
  regression coefficients per (ID, doy); optional repairs; optional TD
predictions

1  climatology        <- BUILD-CLIMATOLOGY(TD, TMIN)
2  bucketed_training  <- BUILD-BUCKET-YEAR-DATASET(TD, TMIN, B)
3  bucketed_clim      <- SHARD-CLIMATOLOGY(climatology, B)
4  coefficients       <- TRAIN-COEFFICIENTS(bucketed_training, bucketed_clim, h,
B)
5  incomplete_doys    <- CHECK-COMPLETENESS(coefficients, Grid)
6  if incomplete_doys is not empty then
7      patch          <- RERUN-FAILED-DOYS(bucketed_training, bucketed_clim,
incomplete_doys, h, B)
8      coefficients   <- PATCH-COEFFICIENTS(coefficients, patch, incomplete_doys)
```

```
9  end if
10 if forecasting requested then
11     bucketed_future_tmin <- SHARD-FUTURE-TMIN(TMIN_future, B)
12     predictions    <- FORECAST-TD(coefficients, bucketed_clim,
bucketed_training,
                                      bucketed_future_tmin, H, B)
13 end if
14 return coefficients, predictions
```

El flujo aprende primero los valores normales estacionales de cada ubicación (climatología, línea 1); reorganiza los archivos crudos en archivos compactos por bucket (líneas 2–3); ajusta las regresiones locales que constituyen la salida científica (línea 4); ejecuta un control de calidad que reentrena solo los días faltantes (líneas 5–9); y, opcionalmente, proyecta el modelo hacia adelante sobre un horizonte futuro (líneas 10–13).

2.5 Algoritmo dominante: entrenamiento de coeficientes

La etapa que domina el costo es el ajuste de las regresiones locales. Para cada `ID` y cada día del año objetivo, se reúne la vecindad circular de días, se eliminan filas incompletas, se ponderan con el kernel tricube y se resuelve la regresión.

Algoritmo 2 — Entrenamiento de coeficientes de anomalía.

```
Algorithm TRAIN-COEFFICIENTS
Input: bucket-year training dataset; bucketed climatology; half-window h; buckets
B
Output: coefficients per (ID, doy)

1  for each bucket b in 0..B-1 pardo do
2      train_b <- read bucket-year training rows for bucket b
3      clim_b  <- read climatology rows for bucket b
4      for each ID i in bucket b pardo do
5          compute TD_anom, TMIN_anom for i (subtract climatology)
6          create lag features (TD_anom_lag1, TD_anom_lag2, TMIN_anom_lag1)
7          for each target day d in 1..366 pardo do
8              N_d <- rows with circular_distance(doy, d) <= h, complete cases
9              if size(N_d) >= minimum_sample_count then
10                 weight(t) <- tricube(circular_distance(doy(t), d) / h)
11                 beta <- weighted_least_squares(N_d)
12                 write coefficients for (ID i, doy d)
13             end if
14         end for
15     end for
16     write coefficient file for bucket b
17 end for
18 return coefficient dataset
```

La vecindad de un ajuste contiene $m = Y(2h+1) = \Theta(Yh)$ observaciones. Como $F = 5$ es constante, un ajuste cuesta $W_{\text{one-fit}} = \Theta(mF^2 + F^3) = \Theta(m) = \Theta(Yh)$. El trabajo total del entrenamiento es, pues, el costo de un ajuste multiplicado por el número de ajustes (N ubicaciones $\times$ D días del año), según la Ecuación 4:

$$W_{\text{train}} = \Theta(N\,D\,Y\,h). \tag{4}$$

En una PRAM ideal que paraleliza también las sumas de las ecuaciones normales dentro de cada regresión, una reducción binaria sobre las m observaciones tiene profundidad $\Theta(\log m) = \Theta(\log(Yh))$. Este término es el *span* de un ajuste, no trabajo adicional. La implementación CPU/Dask utilizada en los experimentos, en cambio, distribuye buckets y regresiones completas entre trabajadores y no paraleliza internamente cada ajuste. Por ello, su tiempo práctico se modela como

$$T^{\text{impl}}_{p,\ \text{train}} = \Theta\left(\frac{NDYh}{\min(p, B)}\right) + T_{\text{E/S}} + T_{\text{planificación}}.$$

2.6 Complejidad global

Sumando las fases dominantes, con $R = NYD$, el trabajo del pipeline completo es

$$W_{\text{pipeline}} = \Theta\left(NYD\log\left(\frac{ND}{B}\right) + NDYh + NQYh + NH\right),$$

donde los términos con Q y H aparecen solo si se ejecutan la reparación y el pronóstico, respectivamente; las pasadas lineales de climatología, fragmentación y verificación quedan absorbidas por los términos mostrados. No se elimina asintóticamente el factor de ordenamiento $\log(ND/B)$: cuando el entrenamiento domina en el régimen observado puede usarse la aproximación $W_{\text{pipeline}} \approx \Theta(NDYh)$, pero esa aproximación se declara de forma explícita. El tiempo paralelo combina las fases distribuidas entre p trabajadores y las etapas de cómputo bucket-paralelas, cuyo paralelismo efectivo es $p_{\text{eff}} = \min(p, B)$. La Tabla 2 recopila los costos por fase.

La expresión final de tiempo paralelo del pipeline es la siguiente; su derivación término a término se presenta en el Apéndice A, subsección «Complejidad global»:

$$T_{p,\ \text{pipeline}}(N) = O\left(\begin{array}{c} \frac{NYD\log(ND/B)}{p} + \frac{NDYh}{\min(p,B)} + \frac{NQYh}{\min(p,B)} \\ + \frac{NH}{\min(p,B)} + H + \log p + \tau_{\text{fit}} \end{array}\right). \tag{5}$$

Aquí $\log p$ representa la profundidad de las reducciones y consolidaciones binarias entre trabajadores. El camino crítico de un ajuste es $\tau_{\text{fit}} = \Theta(\log(Yh))$ para la PRAM fina, o $\Theta(Yh)$ cuando cada ajuste es una tarea secuencial, como en CPU/Dask. Como Y y h se mantienen fijos, τ_{fit} es constante respecto de N y p. La Ecuación 5 no incluye los sobrecostos dependientes del hardware de E/S, planificación y contención de memoria, que se analizan experimentalmente.

Tabla 2: Trabajo y tiempo paralelo por fase del pipeline.

Fase	Trabajo secuencial W	Tiempo paralelo T_p
Construir climatología	$O(R)$	$O(R/p + \log p)$
Dataset bucket-año	$O(R\log(ND/B))$	$O(R\log(ND/B)/p + \log p)$
Entrenar coeficientes	$O(NDYh)$	$O(NDYh/\min(p, B))$
Verificar completitud	$O(ND)$	$O(ND/p + \log p)$
Reparar Q DOYs	$O(NQYh)$	$O(NQYh/\min(p, B))$
Pronóstico (horizonte H)	$O(NH)$	$O(NH/\min(p, B) + H)$

El cuello de botella práctico es $O(NDYh)$; la organización bucket-año reduce la E/S asociada al acceso a los datos, pero no modifica el trabajo estadístico de las regresiones locales.

2.7 Escalabilidad teórica e isoeficiencia

La isoeficiencia responde cómo debe crecer el problema con p para mantener una eficiencia constante. Si un algoritmo se expresa como

$$T_p = \Theta\left(\frac{W}{p} + G(p)\right),$$

entonces

$$E = \frac{T_1}{pT_p} \approx \frac{W}{W + pG(p)}.$$

Al fijar $E = c$, con $0 < c < 1$, se obtiene la condición

$$W = \Theta\ (\ pG(p)\ ),$$

que es la función de isoeficiencia (Kumar & Gupta, 1994). En las derivaciones siguientes se supone $p \leq B$; para sostenerlas al aumentar p también debe cumplirse $B = \Omega(p)$.

2.7.1 Isoeficiencia del entrenamiento

Para el entrenamiento aislado, $W_{\text{train}} = \Theta(NDYh)$ y $G_{\text{train}} = \tau_{\text{fit}}$. Por tanto,

$$E_{\text{train}} = \frac{W_{\text{train}}}{W_{\text{train}} + p\tau_{\text{fit}}},$$

y mantenerla constante requiere

$$NDYh = \Theta(p\tau_{\text{fit}}).$$

Tanto en la PRAM fina, donde $\tau_{\text{fit}} = \Theta(\log(Yh))$, como en la ejecución por ajustes completos, donde $\tau_{\text{fit}} = \Theta(Yh)$, D, Y y h son constantes en el barrido. En consecuencia,

$$\boxed{N_{\text{train}}(p) = \Theta(p).} \tag{6}$$

Esta relación justifica usar $N(p) = N_0 p$ en el escalamiento débil del Trabajo A: al crecer el número de procesos, crece en la misma proporción el número de regresiones independientes.

2.7.2 Isoeficiencia del pipeline completo

Para el pipeline, las reducciones y consolidaciones entre trabajadores aportan $\Theta(\log p)$ y el pronóstico aporta la dependencia secuencial $\Theta(H)$. Así,

$$G_{\text{pipeline}}(p) = \Theta(\log p + H + \tau_{\text{fit}}),$$

y la condición general de isoeficiencia es

$$\boxed{W_{\text{pipeline}}(p) = \Theta(p(\log p + H + \tau_{\text{fit}})).} \tag{7}$$

Si D, Y, h y H son constantes, entonces H y τ_{fit} son constantes y

$$W_{\text{pipeline}}(p) = \Theta(p \log p).$$

Conservando el ordenamiento bucket-año, la relación exacta en función de N es

$$NYD \log\left(\frac{ND}{B}\right) + NDYh + NQYh + NH = \Theta(p(\log p + H + \tau_{\text{fit}})). \tag{8}$$

La solución para $N(p)$ depende entonces de cómo crece B y de qué término domina. En el régimen estudiado, si domina el entrenamiento —o si el ordenamiento se amortiza al reutilizar los buckets preparados—, $W_{\text{pipeline}} = \Theta(NDYh) = \Theta(N)$ y se obtiene el corolario

$$\boxed{N_{\text{pipeline}}(p) = \Theta(p \log p).} \tag{9}$$

Esta condición corresponde al pipeline de extremo a extremo; no debe confundirse con $N_{\text{train}}(p) = \Theta(p)$, que es la condición empleada por el experimento de entrenamiento aislado.

2.7.3 Alcance de la isoeficiencia en GPU

No se estima una función de isoeficiencia GPU en este trabajo porque solo se dispuso de una partición A100 MIG, es decir, $p_{\text{GPU}} = 1$. El barrido de tamaño de lote M permite medir ocupación y utilización de ancho de banda dentro de ese dispositivo, pero no determina cómo debe crecer N al aumentar el número de GPUs. Una isoeficiencia multi-GPU requeriría mediciones con $p_{\text{GPU}} \in \{1, 2, 4, ...\}$ y un modelo explícito del costo de distribución y consolidación entre dispositivos. Sin esos datos, una relación de isoeficiencia GPU sería especulativa; el *roofline* y la utilización de ancho de banda son las métricas apropiadas para el diseño experimental disponible.

2.8 Implementación y ejecución de los experimentos

La implementación se apoya en **Dask** para la planificación de tareas y la paralelización, en CPU (mediante `dask_jobqueue.SLURMCluster`) y, en GPU, ejecutándose en proceso sobre una única partición A100 MIG (sin `dask-cuda`) (Rocklin, 2015). El código fuente y los materiales de reproducibilidad están disponibles en el repositorio institucional del proyecto (Centro Internacional de la Papa, 2026). Conviene distinguir dos niveles de paralelismo. El **nivel externo** (bucket) es el número de buckets que corren a la vez, acotado por $\min(p, B)$: en CPU, p es el número de procesos trabajadores de un hilo ($\approx$ núcleos); en GPU fue un solo dispositivo (la partición A100 MIG disponible). El **nivel interno** (ajuste) es cuántos de los $\approx (N/B)\, D$ ajustes de un bucket se evalúan juntos: en CPU es uno a la vez (BLAS fijado a un hilo), mientras que una GPU ensambla y resuelve del orden de 10^5 sistemas 5×5 en una sola llamada por lotes. Por ello la CPU prefiere **muchos** buckets ($B \geq 4p$) para mantener ocupados los núcleos, mientras que la GPU prefiere buckets **lo bastante grandes** como para llenar cada lote.

2.8.1 Configuración de las corridas

Los parámetros estadísticos se mantuvieron idénticos en los tres trabajos: $h = 11$, kernel tricube, mínimo de 15 muestras y las cuatro características definidas anteriormente. El **Trabajo A** evaluó el entrenamiento en CPU sobre PISCOt v1.2 y subconjuntos de 512, 1024 y 2048 `IDs` del período 1981–2016. El **Trabajo B** caracterizó la GPU y entrenó el dominio de aproximadamente 300 000 `IDs` de PISCOt v1.2 para 1981–2016.

El **Trabajo C** utilizó 20 000 `IDs` y separó temporalmente ajuste y evaluación: los coeficientes de ambas versiones se estimaron únicamente con 1981–2014; después se pronosticó el año 2015 usando TMIN de cada versión y se comparó con la TD observada de 2015. Por tanto, 2015 es un **año de prueba fuera de muestra**: sus valores observados de TD no intervinieron en el cálculo de los coeficientes y se usaron exclusivamente para evaluar RMSE, MAE, sesgo y correlación.

2.8.2 Qué cuenta como un procesador `p`

El eje de escalamiento `p` cuenta **trabajadores independientes**, no hilos, para que $S(p)$ y $E(p)$ signifiquen lo mismo en ambos hardwares. En CPU, un trabajador es un proceso Dask de un

solo hilo (un núcleo, con las bibliotecas BLAS fijadas a un hilo mediante `OMP_NUM_THREADS=1`), y `p` se varía escalando el número de procesos con `dask_jobqueue.SLURMCluster`. En el estudio de GPU, en cambio, el despliegue objetivo (KHIPU) expuso **una única partición A100 MIG** (`a100_3g.20gb`; límite de cuenta `gres/gpu=1`), por lo que **no hubo barrido de `p` en GPU** ($p_{GPU} = 1$ siempre) y reportar $S(p)/E(p)$ sobre GPUs no resulta significativo aquí; la GPU se caracterizó por su **rendimiento y un *roofline*** al crecer el tamaño del problema (eje M), no por un barrido de procesadores.

El paralelismo *interno* de la GPU —los miles de hilos CUDA y su organización en bloques— **no se expone como `p` ni se barre como eje de escalamiento**. La ruta de producción es un kernel CUDA propio (`solve5`: una resolución de Cholesky 5×5 en registros, **un hilo por ajuste**) que recibe los $\approx 10^5$ sistemas 5×5 de un bucket y los reparte entre los multiprocesadores (SM) del dispositivo. El número de hilos por bloque es un parámetro de lanzamiento fijo (128 en este estudio), afinado una sola vez; cambiarlo no altera los resultados, solo el tiempo de reloj. Esa configuración intra-dispositivo se mantiene fija durante todo el estudio y se absorbe en la constante c del modelo $T_p = c\,(W/p_{\text{eff}}) + \text{sobrecosto}$, por lo que no es la variable de escalamiento. (El solucionador *batched* de cuSOLVER vía CuPy se usa solo como oráculo de verificación numérica, no en producción.)

Lo único que controla la *carga* ofrecida a una sola GPU es el **ancho de lote** M: el número de ajustes resueltos por llamada al kernel, igual al número de ajustes por bucket, $M \approx (N/B)\,D$. Como hay **un hilo CUDA por ajuste**, M es el análogo directo del eje Q (número de hilos) de la teoría de *oleadas*: mientras M cabe en los hilos residentes del dispositivo el tiempo es casi plano, pero al superar esa capacidad el trabajo se parte en oleadas sucesivas y el tiempo crece de forma lineal con M. En la partición MIG empleada (`a100_3g.20gb`, 42 SM) el kernel compilado usa 127 registros por hilo, de modo que la presión de registros —y no el límite nominal de 2048 hilos/SM— fija la ocupancia: caben 512 hilos por SM y el dispositivo mantiene $42 \times 512 = 21\,504$ ajustes en vuelo a la vez (véase la subsección de granularidad de oleadas en Resultados). La saturación real aparece, por tanto, por debajo del máximo teórico. Por ello, como estudio complementario al barrido de `p`, se **barre el ancho de lote M** (equivalentemente, se reduce `B`, pues $M \approx (N/B)\,D$) para localizar el punto en que una sola A100 se satura. Ese barrido de tamaño caracteriza el nivel interno de paralelismo sin alterar el eje `p` —que sigue siendo el número de GPUs—, y es independiente del número de hilos por bloque, un detalle de lanzamiento que se afina por separado.

2.8.3 Cantidades medidas

Para un tamaño de problema fijo, con $T(p)$ el tiempo de reloj usando p procesadores, se reportan tres cantidades estándar de rendimiento paralelo (JáJá, 1992; Kumar & Gupta, 1994):

$$\text{Tiempo: } T(p), \qquad \text{Aceleración: } S(p) = \frac{T(1)}{T(p)}, \qquad \text{Eficiencia: } E(p) = \frac{S(p)}{p}.$$

La aceleración ideal es $S(p) = p$ y la eficiencia ideal $E(p) = 1$. En la práctica, la fracción secuencial y los sobrecostos limitan la aceleración al aumentar p, principio formalizado para problemas de tamaño fijo por la ley de Amdahl (Amdahl, 1967). En este pipeline, además, las etapas de entrenamiento son bucket-paralelas con $p_{\text{eff}} = \min(p, B)$; por tanto, aun en el modelo ideal, añadir trabajadores después de $p = B$ no reduce el término dominante. El análisis de escalabilidad debe considerar a la vez el crecimiento del problema, el número de procesadores y el sobrecosto paralelo (Kumar & Gupta, 1994).

2.8.4 Escalamiento fuerte y débil

El **escalamiento fuerte** mantiene fijo el tamaño N y aumenta p: pregunta cuánto se reduce el tiempo de una instancia invariable, por lo que está sujeto al límite de Amdahl (Amdahl, 1967). Se fijó un subconjunto representativo de `IDs` y se varió $p \in \{1, 2, 4, 8, 16, 32\}$ procesos en CPU (límite de cuenta `cpu=32` en KHIPU), registrando $T(p)$, $S(p)$ y $E(p)$ para la fase dominante de entrenamiento. La aceleración puede ser cercana a lineal solo mientras haya suficiente paralelismo ($p \leq B$) y los sobrecostos o recursos compartidos no dominen. En GPU, al disponerse de un solo dispositivo, no hubo barrido de escalamiento fuerte: la GPU se evaluó mediante el *roofline* y el barrido de M ya descritos.

El **escalamiento débil** aumenta el problema junto con los procesadores para mantener constante la carga asignada a cada trabajador, criterio relacionado con la aceleración de problema escalado de Gustafson (Gustafson, 1988). Sea $n(p)$ el número de `IDs` entrenados. Como el Trabajo A mide solo el entrenamiento, se aplica la condición $N_{\text{train}}(p) = \Theta(p)$ derivada en la Sección 2.7.1: el protocolo usa $n(p) = n_0 p$ y reporta

$$E_{\text{weak}}(p) = \frac{T(1, n_0)}{T(p, n_0 p)}.$$

Esta métrica compara ejecuciones con tamaños distintos y sus desviaciones respecto de 1 cuantifican E/S, planificación y contención de memoria (Kumar & Gupta, 1994). La relación $N_{\text{pipeline}}(p) = \Theta(p \log p)$ corresponde, bajo los supuestos indicados en la Sección 2.7.2, al pipeline completo y no a este experimento de entrenamiento aislado. Por su parte, las curvas de escalamiento fuerte mantienen n constante y se interpretan con los modelos de referencia de la Sección 3.1.

2.8.5 Controles y reportes

Para garantizar mediciones reproducibles y justas: la línea base $T(1)$ usó un único trabajador con un hilo y los pools de BLAS fijados a 1; el número de buckets se mantuvo en $B \geq 4\,p_{\max}$ durante todo el barrido; cada punto se ejecutó varias veces y se reportó la mediana; el almacenamiento temporal fue a disco local rápido del nodo; y se distinguió entre caché caliente y fría. El protocolo completo se ejecutó sobre **PISCOt v1.1 y v1.2**, comparando tanto el rendimiento (curvas $T(p)$, $S(p)$, $E(p)$) como la ciencia (distribuciones de los coeficientes ajustados y de la habilidad predictiva), para evaluar cómo la versión del dataset afecta al modelo estimado con independencia del tiempo de ejecución.

3 Resultados

El protocolo descrito se ejecutó en su totalidad sobre el clúster **KHIPU** (UTEC; OpenHPC + Slurm), en tres trabajos: **Trabajo A** (CPU: escalamiento fuerte y débil del entrenamiento sobre un subconjunto de `IDs`, v1.2), **Trabajo B** (GPU: *roofline* y modelo de 300 000 puntos, v1.2) y **Trabajo C** (comparación de habilidad predictiva v1.1 vs v1.2 sobre el año de prueba fuera de muestra 2015). Todas las corridas de entrenamiento son numéricamente equivalentes en CPU y GPU (verificado a $\max|\Delta| \approx 3 \times 10^{-15}$), por lo que la elección de hardware afecta la **velocidad**, no el resultado científico.

Para interpretar las mediciones se emplean modelos de referencia distintos para CPU y GPU, porque cada hardware expone el paralelismo en un eje diferente: la CPU barre **procesadores** $\boldsymbol{p}$ a tamaño fijo, mientras que la única GPU barre el **tamaño del problema** $\boldsymbol{M}$ con $p = 1$. Estos modelos experimentales son distintos de las funciones de isoeficiencia derivadas para el entrenamiento y para el pipeline completo.

3.1 Modelos de referencia para las curvas experimentales

3.1.1 CPU: modelo de contención de ancho de banda de memoria

La curva de CPU describe **escalamiento fuerte con tamaño fijo**: sea $n = N$ constante y, como D, Y y h también permanecen fijos durante el barrido, el trabajo $W = nDYh$ es constante. No se trata de otra derivación de isoeficiencia, sino de un modelo del recurso compartido que explica las mediciones. Si la pérdida de eficiencia se debiera a un sobrecosto fijo aditivo, este se diluiría al crecer n; sin embargo, las curvas medidas para tres valores de n casi se superponen. Esto señala un costo consumido en proporción al trabajo y compartido por los núcleos: el acceso a memoria.

Si cada unidad de trabajo necesita un tiempo de cómputo t_c y transfiere q bytes a través de un bus con ancho de banda β, el tiempo con p procesadores es

$$T_p = \underbrace{\frac{Wt_c}{p}}_{\text{cómputo, repartido}} + \underbrace{\frac{Wq}{\beta}}_{\text{bus, no repartido}} . \tag{10}$$

Para un procesador, $T_1 = W(t_c + q/\beta)$. Definiendo $\gamma = (q/\beta)/t_c$, la aceleración y la eficiencia teóricas son

$$S(p) = \frac{T_1}{T_p} = \frac{p(1+\gamma)}{1+\gamma p}, \qquad E(p) = \frac{T_1}{pT_p} = \frac{1+\gamma}{1+\gamma p}. \tag{11}$$

Aunque la derivación mantiene n constante, el trabajo $W = nDYh$ **se cancela** en el cociente: un único γ describe todos los tamaños, coherente con la superposición observada. Asintóticamente, para $p \gg 1/\gamma$, $E(p) = O(1/p)$ y la aceleración satura en $S(\infty) = 1 + 1/\gamma$. Un ajuste por mínimos cuadrados sobre los tres tamaños dio $\gamma \approx 0.063$ (el costo de acceso a memoria equivale aproximadamente al 6 % del tiempo de cómputo por unidad de trabajo) y $S(\infty) \approx 17$. Como W se cancela, aumentar N no recupera la eficiencia perdida en un único bus compartido; esta limitación del hardware real no contradice la isoeficiencia PRAM del entrenamiento.

3.1.2 GPU: la curva teórica cuando $p = 1$

El estudio de GPU no tiene eje de procesadores ($p_{GPU} = 1$): el eje es el tamaño $M = N \cdot 366$ ajustes. Por ello no puede calcularse aceleración, eficiencia ni isoeficiencia respecto del número de GPUs. Como todas las etapas están muy por debajo del vértice del *roofline* (intensidad aritmética $I \approx 0.08$–0.6 FLOP/byte frente a un vértice ≈ 6.6), cada etapa está **limitada por ancho de banda** y su tiempo tiene como cota inferior el tiempo de mover sus bytes (Ecuación 12):

$$T_{\text{bound}}(M) = \frac{Q(M)}{\beta}, \qquad \eta(M) = \frac{T_{\text{bound}}(M)}{T_{\text{med}}(M)} = \frac{\text{GB/s alcanzados}}{\beta} \in (0, 1]. \tag{12}$$

La **utilización de ancho de banda** $\eta(M)$ es el análogo de la eficiencia para una sola GPU: con un costo fijo de lanzamiento t_0, $T(M) \approx t_0 + Q(M)/\beta$ y $\eta(M) \to 1$ al crecer M (el lote llena el dispositivo). Es la imagen especular del caso CPU: allí la eficiencia *decae* como $O(1/p)$ por contención; aquí la utilización *satura* al crecer el lote, y por eso la GPU prefiere buckets **grandes**. Una isoeficiencia GPU solo tendría sentido en un experimento futuro con varias GPUs y un modelo del costo de comunicación entre ellas; para la única partición MIG disponible, el *roofline* es la referencia teórica pertinente.

3.2 Granularidad de oleadas: cuántos ajustes corren a la vez

El *solve* por lotes no ejecuta los M ajustes simultáneamente; el hardware los corre en **oleadas** (*waves*), y las mediciones bastan para contarlas:

1. El kernel usa **un hilo por ajuste** y necesita **127 registros por hilo** (un *registro* es el almacenamiento privado y rápido de un hilo; aquí guarda el sistema 5×5 y los intermedios de Cholesky). Este número lo elige el compilador de CUDA y se registra en la columna `num_regs` de `gpu_kernel.csv`.
2. Los hilos se planifican en **warps** de 32, así que un warp necesita $32 \times 127 \approx 4\,064$ registros, redondeados a $4\,096$.
3. Cada SM posee $65\,536$ registros, de modo que $65\,536/4\,096 = 16$ warps $= 512$ hilos caben por SM (limitado por registros al 25 % del máximo nominal de 2048 hilos/SM).
4. La partición MIG tiene **42 SM**, así que el dispositivo mantiene $42 \times 512 = 21\,504$ ajustes en vuelo, y oleadas $(M) = \lceil M/21\,504 \rceil$.

Tabla 3: Oleadas del kernel *solve* en la partición A100 MIG.

M (ajustes)	oleadas	tiempo *solve* (ms)	µs por oleada
23 424	2	0.0266	13.3
107 238	5	0.0727	14.5
187 392	9	0.1065	11.8
374 784	18	0.1915	10.6
800 000	38	0.3835	10.1

El tiempo por oleada converge a ≈ 10 µs al crecer M (los valores mayores a M pequeño son el costo de lanzamiento t_0), y 21 504 ajustes cada 10 µs reproducen el rendimiento saturado medido de $\approx 2 \times 10^9$ ajustes/s. A escala completa, el dominio de 300 000 `IDs` ($M_{\text{total}} \approx 1.1 \times 10^8$ ajustes) corrió unas **5 150 oleadas** de *solve* en total, ≈ 5 por bucket de $B = 1024$. No hace falta un gráfico de "oleadas vs M": es la recta $M/21\,504$ por construcción; la evidencia visual está en la curva de tiempo vs M, que crece linealmente por encima de una oleada.

3.3 Resultados en CPU (Trabajo A)

Se midió el escalamiento fuerte del entrenamiento en tres tamaños de problema (512 / 1024 / 2048 `IDs`), con $p \in \{1, 2, 4, 8, 16, 32\}$ y $B \geq 4p$. La Tabla 4 recoge la eficiencia $E(p)$: las tres curvas son casi indistinguibles, la firma de una carga **independiente del tamaño** y limitada por ancho de banda de memoria; la última columna es la curva teórica ajustada de la Ecuación 11 con $\gamma = 0.063$.

Tabla 4: Eficiencia $E(p)$ del entrenamiento por tamaño de problema, con la curva teórica del modelo de contención.

p	$E(p)$, N=512	$E(p)$, N=1024	$E(p)$, N=2048	$E(p)$ teórico
1	1.000	1.000	1.000	1.000
2	0.939	0.917	0.912	0.944
4	0.830	0.826	0.824	0.849
8	0.709	0.704	0.699	0.707
16	0.562	0.560	0.557	0.529
32	0.345	0.342	0.337	0.352

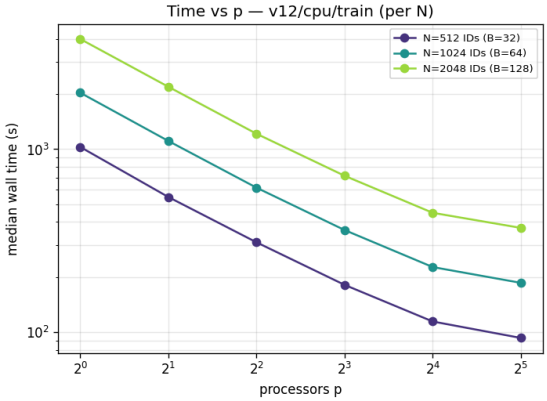


Figura 1: Tiempo de pared del entrenamiento frente al número de procesos para 512, 1024 y 2048 `IDs`.

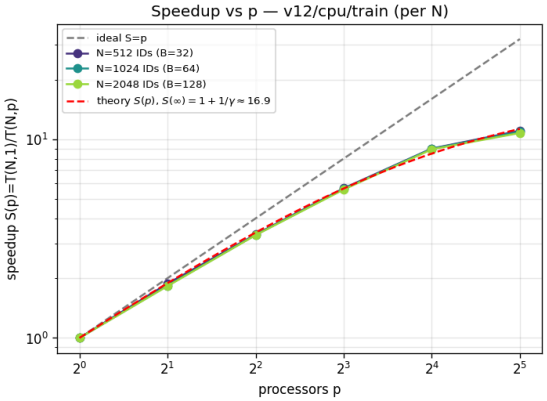


Figura 2: Aceleración observada frente al número de procesos para los tres tamaños de problema.

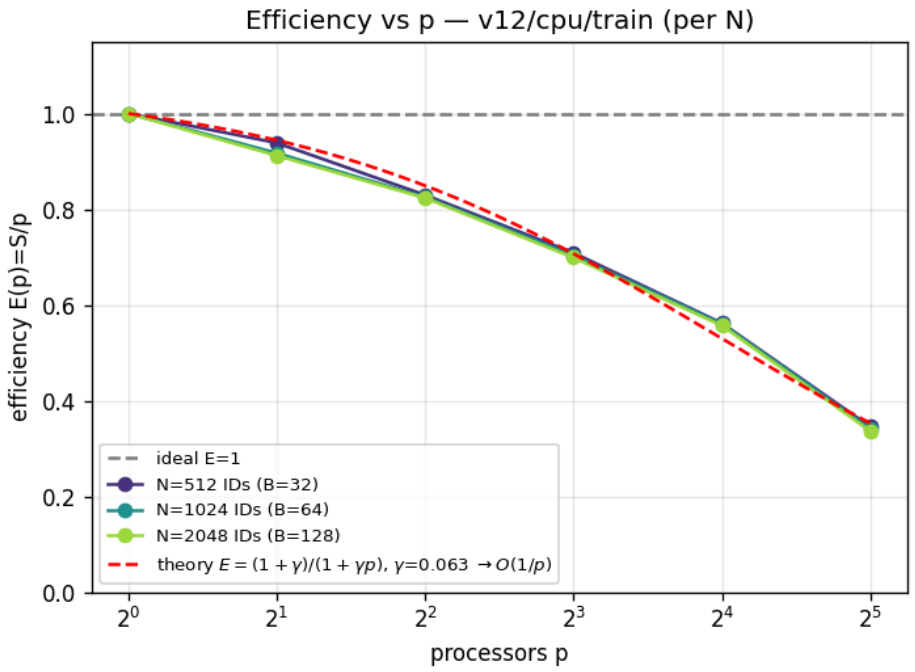


Figura 3: Eficiencia paralela frente al número de procesos. Las curvas de los tres tamaños casi se superponen; la línea roja discontinua representa el modelo de contención de memoria ajustado $E(p) = (1+\gamma)/(1+\gamma p)$.

Con $p = 1$, el costo observado fue 2.01, 1.99 y 1.95 s/`ID` para 512, 1024 y 2048 `ID`s, respectivamente. Por tanto, al mantener fijo el número de procesadores, duplicar los `ID`s casi duplicó el tiempo de pared: el costo crece aproximadamente en proporción a N, coherente con $T_1 = O(NDYh)$ cuando D, Y y h permanecen fijos. La eficiencia se mantuvo alta hasta ~4–8 núcleos y luego cayó —no por desbalance ni sobrecosto fijo (un problema mayor los diluiría), sino porque todos los núcleos compiten por el **mismo bus de memoria**—: la eficiencia medida a 32 núcleos (≈ 0.35) es la firma de una carga **limitada por ancho de banda**. El modelo $E(p) = (1+\gamma)/(1+\gamma p)$ con $\gamma \approx 0.063$ reproduce las mediciones dentro de pocos puntos porcentuales. El punto óptimo práctico es $\boldsymbol{p \approx 8}$ (eficiencia ≈ 0.70); 32 núcleos dan el mejor tiempo de pared pero a aproximadamente un tercio de la eficiencia, con una aceleración que sigue trepando hacia el techo $S(\infty) \approx 17$ (medida $\approx 11\times$ a $p = 32$; Figura 2). La Figura 3 muestra que el mismo modelo de contención explica los tres tamaños del problema.

3.4 Resultados en GPU (Trabajo B)

Sobre una sola partición A100 MIG (`a100_3g.20gb`), el pipeline de entrenamiento tiene tres etapas —**assemble** (dispersa filas en estadísticos suficientes 5×5), **convolve** ($2h+1$ desplazamientos circulares ponderados sobre el día del año) y **solve** (el kernel Cholesky 5×5 de un hilo por ajuste)—. La Tabla 5 resume el desempeño de un dispositivo.

Tabla 5: Desempeño de la GPU en un solo dispositivo.

Magnitud	Valor
Ancho de banda HBM medido	**633 GB/s**
Pico FP64 (partición)	4200 GFLOPS (vértice AI ≈ 6.6)
Mejor hilos/bloque	32 (1.81×10^9 ajustes/s)
Rendimiento del *solve*	hasta 2.09×10^9 ajustes/s
Intensidad aritmética por etapa	assemble 0.11 · convolve 0.08 · solve 0.61 (todas $\ll 6.6$)
Entrenamiento completo 300 000 puntos	**879.7 s ($\approx$ 14.7 min)**

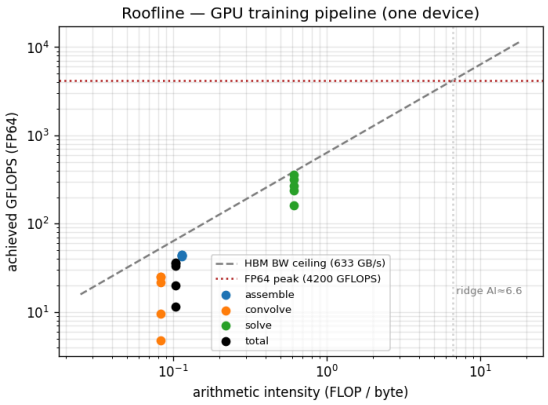


Figura 4: *Roofline* del pipeline de entrenamiento. Todas las etapas quedan por debajo del techo de cómputo y en la región limitada por ancho de banda.

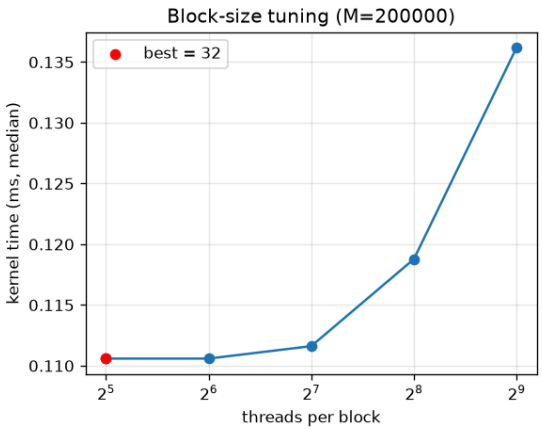


Figura 5: Ajuste del tamaño de bloque del kernel *solve*; 32 hilos por bloque fue la mejor configuración en la partición evaluada.

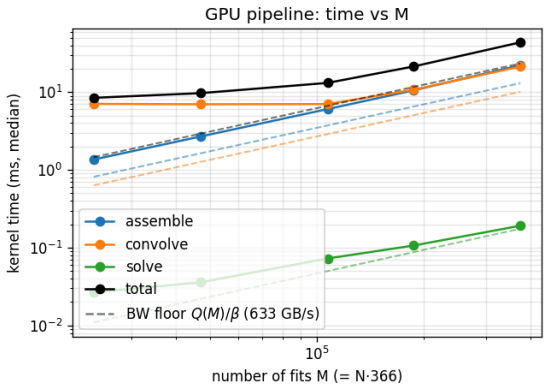


Figura 6: Tiempo de cada etapa frente al tamaño del problema M. Las líneas discontinuas son el piso teórico de ancho de banda $T_{\text{bound}}(M) = Q(M)/\beta$.

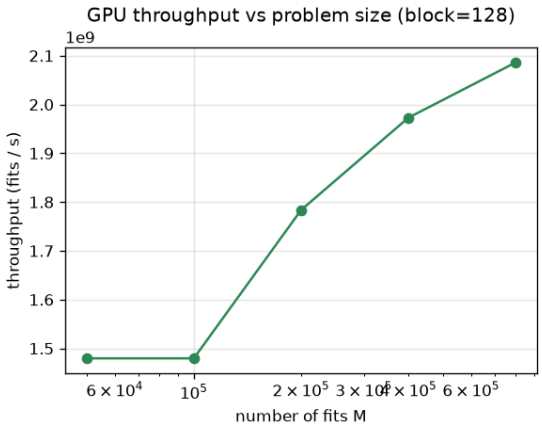


Figura 7: Rendimiento del kernel *solve* frente a M: la tasa de ajustes por segundo aumenta hasta saturar el dispositivo.

Todas las etapas se sitúan muy por debajo del vértice del *roofline*, por lo que el pipeline está **limitado por ancho de banda, no por cómputo** —lo esperable para estos sistemas lineales diminutos—. Aun así, el rendimiento bruto es enorme ($\approx 2 \times 10^9$ ajustes/s) y el dominio

completo de 300 000 puntos se entrenó en menos de 15 minutos. La curva de tiempo vs M crece linealmente por encima de una oleada, y su cota teórica es el piso de ancho de banda $T_{\text{bound}} = Q(M)/\beta$ de la Ecuación 12, como se observa en la Figura 6.

CPU frente a GPU. Sobre una base de costo por `ID` entrenado: CPU 1 núcleo ≈ 2.0 s/`ID` (1 ×), CPU 32 núcleos ≈ 0.18 s/`ID` ($\approx 11\times$) y GPU ≈ 0.0029 s/`ID` ($\approx 670\times$). Es decir, para el **entrenamiento**, la GPU fue $\approx 60\times$ más rápida que un nodo CPU de 32 núcleos plenamente cargado. El **pronóstico**, en cambio, es autorregresivo (cada día predicho alimenta los rezagos del siguiente), inherentemente secuencial y en Python/pandas puro, por lo que la GPU **no lo acelera**; solo queda paralelizarlo entre `ID`s independientes en CPU.

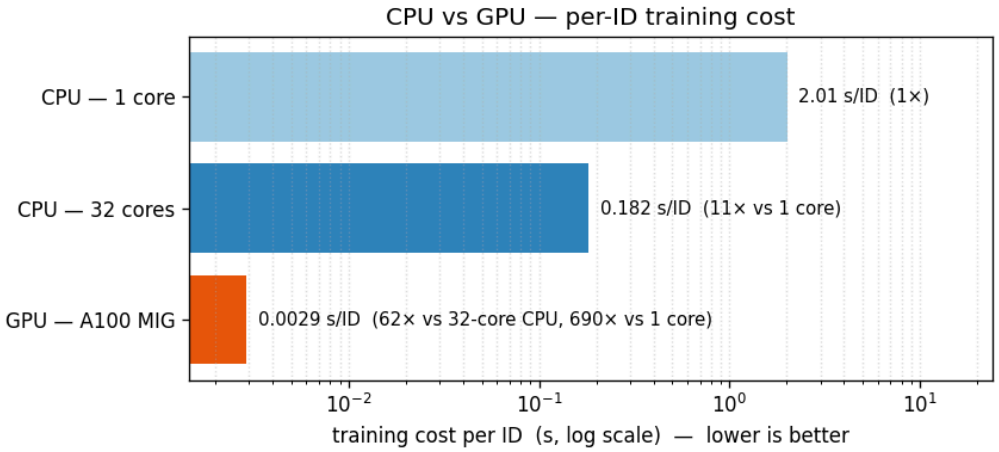


Figura 8: Costo de entrenamiento por `ID` en escala logarítmica (menor es mejor). La comparación usa el costo por `ID` porque los *benchmarks* de CPU y GPU se ejecutaron con tamaños de problema distintos.

3.5 Comparación PISCOt v1.1 vs v1.2 (Trabajo C)

Sobre el pronóstico fuera de muestra de 2015 (20 000 `ID`s, ~7.3 millones de valores diarios evaluados contra la TD observada), ambas versiones producen modelos muy similares (Tabla 6).

Tabla 6: Habilidad del pronóstico frente a la TD observada; ΔRMSE (v1.2−v1.1) = +0.0089 (~0.8 %).

Corrida	RMSE	MAE	Sesgo (pred−obs)	Pearson r	Coseno
v1.1	**1.127**	**0.844**	+0.443	**0.9823**	0.9926
v1.2	1.136	0.867	**+0.143**	0.9787	0.9921

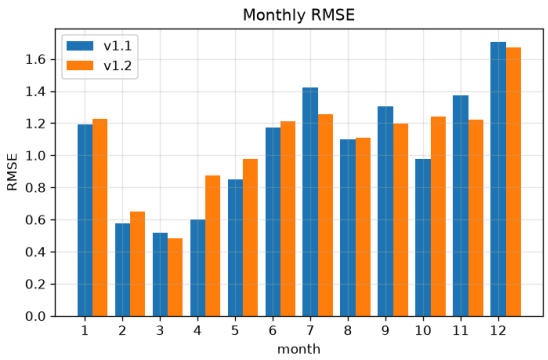


Figura 9: RMSE mensual del pronóstico fuera de muestra, PISCOt v1.1 frente a v1.2.

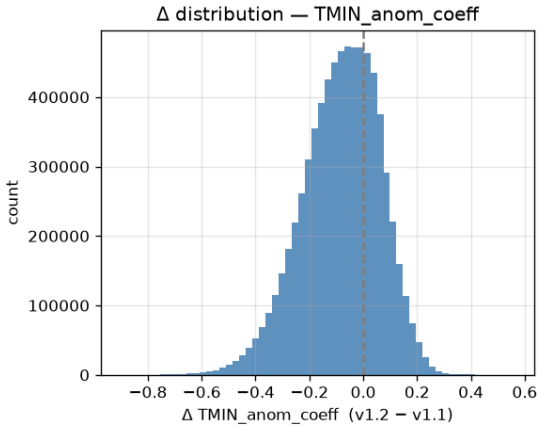


Figura 10: Distribución de la diferencia v1.2−v1.1 del coeficiente de la anomalía de TMIN, la mayor diferencia entre los modelos.

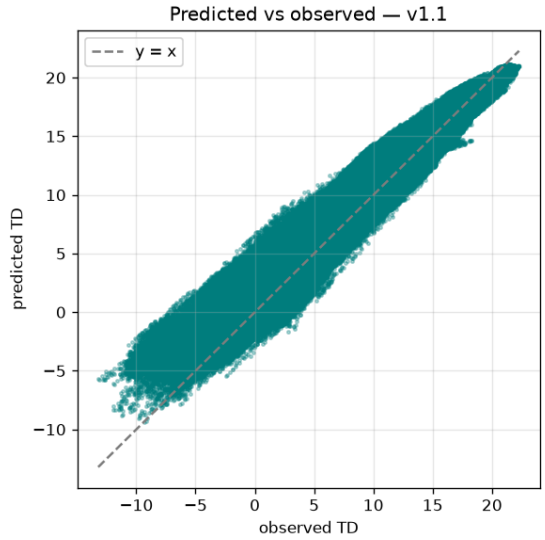


Figura 11: Temperatura de punto de rocío predicha frente a la observada con PISCOt v1.1.

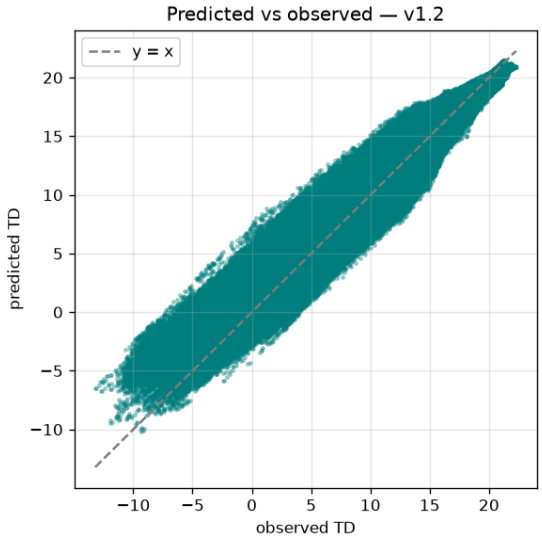


Figura 12: Temperatura de punto de rocío predicha frente a la observada con PISCOt v1.2.

Ambas versiones dan idéntica cobertura (7.32 M ajustes) y modelos muy parecidos: la mayor diferencia está en la **sensibilidad a la anomalía de TMIN** (`TMIN_anom_coeff`: v1.1 = 0.452 vs v1.2 = 0.370; la de menor correlación, $r = 0.68$), y v1.2 se apoya algo más en la persistencia autorregresiva (`TD_anom_lag1` 0.81 vs 0.78). La calidad de ajuste es casi idéntica (R^2 0.787 vs 0.778) y las predicciones de ambas versiones concuerdan estrechamente entre sí (RMSE 0.61, r 0.9946), divergiendo más en septiembre–octubre. En resumen, **v1.1 tiene un error marginalmente menor** (RMSE 1.127 vs 1.136, ~0.8 %) mientras que **v1.2 está mucho mejor centrada** (sesgo +0.14 vs +0.44): prácticamente un empate en magnitud, con un compromiso real entre dispersión (mejor en v1.1) y sesgo (mejor en v1.2). Cuál versión es "mejor" depende de esa prioridad y **debe validarse con un experto temático** sobre una evaluación multianual de dominio completo (esta es una muestra de 20 000 `IDs` para un único año de pronóstico).

3.6 Recomendación

Los resultados confirman punto por punto el modelo teórico. La recomendación práctica es **usar cada hardware según la etapa**: la GPU para el **entrenamiento** (etapa de mayor carga y masivamente paralela; una sola partición A100 MIG entrena el dominio de 300 000 puntos en ~15 min) y la CPU para el **pronóstico** (recurrencia secuencial que la GPU no acelera, paralelizable en cambio entre `ID`s). Si no hay GPU, el entrenamiento corre en CPU con **~8 núcleos** para la mejor eficiencia (≈ 0.70), o hasta 32 para el mejor tiempo de pared (eficiencia ≈ 0.34, limitada por ancho de banda de memoria).

Como mejora futura para CPU, se propone distribuir el entrenamiento entre varios nodos mediante **MPI**, asignando a cada proceso un conjunto disjunto de buckets de `ID`. A diferencia de añadir más procesos dentro de un mismo nodo, la ejecución multinodo permitiría aprovechar el ancho de banda agregado de controladores de memoria físicamente independientes. Esta estrategia es adecuada porque los ajustes correspondientes a distintos `ID`s no presentan dependencias entre sí.

El flujo MPI propuesto tendría tres etapas:

1. **Distribución (`scatter`).** El proceso raíz divide la lista de buckets entre los procesos MPI. Puede emplearse `MPI_Scatterv` para distribuir cantidades distintas de identificadores o metadatos. No se recomienda enviar todas las observaciones mediante MPI, porque el proceso raíz y la red podrían convertirse en cuellos de botella. Cada proceso debería leer directamente los archivos correspondientes a sus buckets.
2. **Cómputo local con afinidad NUMA.** Cada proceso entrena únicamente los `ID`s asignados. Se propone ejecutar un proceso MPI por dominio NUMA y fijar sus núcleos y asignaciones de memoria al mismo socket. Esto reduce los accesos a memoria remota y las migraciones entre sockets. Dentro de cada dominio NUMA se utilizaría un número moderado de trabajadores para evitar volver a saturar el bus de memoria.
3. **Consolidación (`gather`).** Cada proceso produce coeficientes para conjuntos disjuntos de `ID`. La alternativa más escalable es que cada proceso escriba sus archivos de salida y que el proceso raíz reúna solamente métricas, contadores y un manifiesto mediante `MPI_Gather` o `MPI_Reduce`. Si fuera indispensable construir una tabla única en la memoria del proceso raíz, podría utilizarse `MPI_Gatherv`, aunque aumentaría la comunicación y el consumo de memoria del proceso raíz.

No se requiere `MPI_Allgather`, porque ningún proceso necesita recibir los coeficientes calculados por todos los demás. Utilizarlo replicaría el modelo completo en cada proceso y multiplicaría el consumo de memoria. Solo sería necesario si todos los procesos tuvieran que consultar el modelo completo. En este pipeline, el entrenamiento y el pronóstico pueden mantener la misma partición por `ID`, por lo que cada proceso necesita únicamente sus propios coeficientes.

La evaluación de esta mejora debería medir por separado los tiempos de distribución, lectura, entrenamiento y consolidación. Esto permitiría determinar si el ancho de banda agregado de los nodos compensa el costo de comunicación y si el sistema de archivos compartido se convierte en el nuevo cuello de botella.

4 Conclusiones

El trabajo implementó y evaluó una regresión lineal local ponderada para estimar la temperatura de punto de rocío a partir de TMIN sobre la grilla PISCOt. La formulación por anomalías, con

$h = 11$ y cuatro predictores temporales, permitió producir coeficientes para cada `(ID, doy)` y mantener resultados numéricamente equivalentes entre CPU y GPU.

El análisis PRAM establece que la isoeficiencia del entrenamiento es $N_{\text{train}}(p) = \Theta(p)$: idealmente, aumentar el número de `ID`s en la misma proporción que los procesos mantiene constante el trabajo por proceso. Sin embargo, esta condición supone que la capacidad de los recursos necesarios también crece con p. El entrenamiento es intensivo en movimiento de datos y, en el nodo CPU evaluado, todos los procesos comparten un ancho de banda de memoria prácticamente fijo; por ello, el término de memoria Wq/β no se divide entre los trabajadores y aumentar N no recupera la eficiencia. En consecuencia, la eficiencia observada decae asintóticamente como $E(p) = \Theta(1/p)$: es cercana a 0.70 con ocho procesos, pero cae a aproximadamente 0.34 con 32, aunque el tiempo de pared continúa disminuyendo. Así, $N_{\text{train}}(p) = \Theta(p)$ describe la escalabilidad algorítmica ideal, mientras que $E(p) = \Theta(1/p)$ evidencia el límite impuesto por la memoria compartida.

En la GPU, las etapas también están limitadas por movimiento de datos; sin embargo, el paralelismo por lotes permitió entrenar el dominio de aproximadamente 300 000 puntos en 879.7 s, cerca de 14.7 min. Por ello, la GPU es la opción preferida para el entrenamiento y la CPU para el pronóstico autorregresivo.

En la evaluación fuera de muestra de 2015, PISCOt v1.1 obtuvo un RMSE ligeramente menor que v1.2 (1.127 frente a 1.136), mientras que v1.2 redujo considerablemente el sesgo positivo (+0.143 frente a +0.443). La diferencia de RMSE fue de solo ~0.8 %, por lo que estos resultados no justifican una elección definitiva entre versiones. Se requiere ampliar la evaluación a más años y al dominio completo, además de analizar el comportamiento por región climática.

Finalmente, la principal línea de mejora computacional es combinar MPI multinodo con afinidad NUMA y partición persistente por `ID`. Esta alternativa permitiría utilizar el ancho de banda de memoria agregado de varios nodos sin replicar innecesariamente los coeficientes mediante `MPI_Allgather`; su beneficio deberá comprobarse incluyendo los costos de E/S y comunicación.

5 Contribución

Nombre	Porcentaje
Piero Palacios Bernuy	100%
Estuardo Oliver Campos	100%
Diego Sánchez Salazar	100%
Gianmarco Mejía	100%

Referencias

Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998). *Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements* [FAO Irrigation and Drainage Paper 56]. Food and Agriculture Organization of the United Nations.

Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, 483-485. https://doi.org/10.1145/1465482.1465560

Centro Internacional de la Papa. (2026). *tdew_estimation: Estimación de la temperatura de punto de rocío*. Repositorio de GitHub. https://github.com/CENTRO-INTERNACIONAL-DE-LA-PAPA/tdew_estimation

Cleveland, W. S. (1979). Robust Locally Weighted Regression and Smoothing Scatterplots. *Journal of the American Statistical Association*, *74*(368), 829-836. https://doi.org/10.1080/01621459.1979.10481038

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

Fan, J., & Gijbels, I. (1996). *Local Polynomial Modelling and Its Applications*. Chapman & Hall/CRC.

Gustafson, J. L. (1988). Reevaluating Amdahl's Law. *Communications of the ACM*, *31*(5), 532-533. https://doi.org/10.1145/42411.42415

Huber, L., & Gillespie, T. J. (1992). Modeling Leaf Wetness in Relation to Plant Disease Epidemiology. *Annual Review of Phytopathology*, *30*, 553-577. https://doi.org/10.1146/annurev.py.30.090192.003005

Huerta, A., Aybar, C., Imfeld, N., Correa, K., Felipe-Obando, O., Rau, P., Drenkhan, F., & Lavado-Casimiro, W. (2023). High-Resolution Grids of Daily Air Temperature for Peru — the New PISCOt v1.2 Dataset. *Scientific Data*, *10*, 847. https://doi.org/10.1038/s41597-023--02777-w

Hwang, J., Orenstein, P., Cohen, J., Pfeiffer, K., & Mackey, L. (2019). Improving Subseasonal Forecasting in the Western U.S. with Machine Learning. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*, 2325-2335. https://doi.org/10.1145/3292500.3330674

JáJá, J. (1992). *An Introduction to Parallel Algorithms*. Addison-Wesley.

Kumar, V., & Gupta, A. (1994). Analyzing Scalability of Parallel Algorithms and Architectures. *Journal of Parallel and Distributed Computing*, *22*(3), 379-391. https://doi.org/10.1006/jpdc.1994.1099

Narouei-Khandan, H. A., Shakya, S. K., Garrett, K. A., Goss, E. M., Dufault, N. S., Andrade-Piedra, J. L., Asseng, S., Wallach, D., & Bruggen, A. H. C. van. (2020). BLIGHTSIM: A New Potato Late Blight Model Simulating the Response of Phytophthora infestans to Diurnal Temperature and Humidity Fluctuations in Relation to Climate Change. *Pathogens*, *9*(8), 659. https://doi.org/10.3390/pathogens9080659

Rocklin, M. (2015). Dask: Parallel Computation with Blocked Algorithms and Task Scheduling. *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, 126-132. https://doi.org/10.25080/Majora-7b98e3ed-013

Sentelhas, P. C., Dalla Marta, A., Orlandini, S., Santos, E. A., Gillespie, T. J., & Gleason, M. L. (2008). Suitability of Relative Humidity as an Estimator of Leaf Wetness Duration. *Agricultural and Forest Meteorology*, *148*(3), 392-400. https://doi.org/10.1016/j.agrformet.2007.09.011

Apéndice A. Pseudocódigo PRAM y análisis de complejidad completos

Este apéndice reproduce de forma íntegra el documento técnico de pseudocódigo en el modelo PRAM y el análisis de complejidad del pipeline. Detalla los nueve algoritmos (climatología, construcción del dataset bucket-año, fragmentación, entrenamiento de coeficientes, verificación de completitud, reparación, parchado y pronóstico), con sus costos de trabajo secuencial W, tiempo paralelo T_p y span T_∞, así como la interpretación para cómputo de alto desempeño (HPC) y la metodología experimental completa. Las secciones del apéndice se numeran A.1, A.2, …

A.1 Propósito

Este documento describe el algoritmo completo utilizado para estimar la temperatura diaria de punto de rocío (`TD`) a partir de la temperatura mínima (`TMIN`) para muchas ubicaciones espaciales (`ID`). El diseño sigue una visión al estilo PRAM: las operaciones independientes se marcan con `pardo`.

El proyecto utiliza un modelo de regresión lineal local sobre anomalías:

$$TD_{anom}(t) = TD(t) - TD_{clim}(ID, doy)$$

$$TMIN_{anom}(t) = TMIN(t) - TMIN_{clim}(ID, doy)$$

Para cada `ID` espacial y cada día del año `doy`, el modelo ajusta una regresión lineal ponderada usando la anomalía de `TMIN` y variables de anomalía rezagadas.

Por qué anomalías, en términos sencillos. La temperatura cruda tiene un ciclo estacional fuerte y predecible: cada ubicación es más cálida en verano y más fría en invierno. Si modeláramos `TD` cruda contra `TMIN` cruda, la mayor parte de la relación aparente sería simplemente "ambas siguen las estaciones", no el vínculo físico que nos interesa. Por eso primero restamos el valor *típico* de cada ubicación para ese día del año (su **climatología**), dejando la **anomalía**: cuánto está hoy por encima o por debajo de lo normal. La regresión aprende entonces el acoplamiento genuino día a día entre el punto de rocío y la temperatura mínima, habiendo eliminado la estación. Ajustamos una regresión pequeña *separada* para cada `(ID, doy)`, de modo que la relación pueda cambiar por ubicación y por época del año, y tomamos fuerza prestada de los días vecinos del año (una ventana local de semianchura `h`) para que cada ajuste tenga suficientes muestras. Los productos finales son: una tabla de climatología, los coeficientes de regresión ajustados por `(ID, doy)` y, opcionalmente, valores pronosticados de `TD`.

A.2 Modelo PRAM

PRAM (Máquina de Acceso Aleatorio Paralela, por sus siglas en inglés) es el modelo idealizado estándar para razonar sobre algoritmos paralelos: muchos procesadores comparten una memoria, y contamos cómo se reparte el trabajo entre ellos. Usamos la variante **CREW** ("Lectura Concurrente, Escritura Exclusiva"), que se ajusta exactamente a este pipeline:

- **Lectura concurrente**: muchos procesadores pueden leer las *mismas* tablas de entrada compartidas a la vez (p. ej., cada trabajador puede leer la climatología). Las lecturas no entran en conflicto.
- **Escritura exclusiva**: no hay dos procesadores que escriban en la *misma* ubicación de salida. Aquí esto se garantiza de forma natural, cada trabajador es dueño de un bucket / ID / día del año de salida disjunto, así que nunca hay colisiones de escritura que coordinar.

- **Reducciones** (sumas, medias, conteos) se tratan como reducciones paralelas: combinar `m` valores cuesta aproximadamente `log m` pasos en lugar de `m`, porque los valores se suman por pares ascendiendo por un árbol.
- **`pardo`** marca un bucle cuyas iteraciones son independientes y, por tanto, pueden ejecutarse en paralelo. Es la anotación central de este documento: dondequiera que aparezca `pardo`, no hay dependencia entre iteraciones, así que un planificador es libre de repartirlas entre todos los trabajadores disponibles.

El diseño bucket-año es importante porque evita que 300 mil tareas independientes por ID escaneen repetidamente los mismos archivos parquet históricos. En lugar de que cada ID vuelva a abrir cada archivo mensual, los datos se agrupan una sola vez en `B` buckets, y cada trabajador lee los archivos compactos de su bucket una única vez, convirtiendo una tormenta de lecturas diminutas y repetidas en unas pocas lecturas secuenciales grandes.

A.3 Símbolos

Símbolo	Significado
`N`	Número de ubicaciones espaciales, aproximadamente 300 000 IDs
`Y`	Número de años de entrenamiento, aproximadamente 40
`D`	Días por año, como máximo 366
`T`	Total de días por ID, `T = Y * D`
`R`	Total de filas por variable climática, `R = N * T`
`B`	Número de buckets de ID, por ejemplo 1024
`p`	Número de procesadores o ranuras de trabajador disponibles
`h`	Semiventana local de DOY, por defecto `h = 11`
`W`	Muestras por regresión local, `W = O(Y * h)`
`F`	Número de características de regresión más el intercepto, constante aquí
`Q`	Número de DOYs fallidos a reparar, `Q <= D`
`H`	Horizonte de pronóstico en días

En este proyecto, `D`, `h` y `F` son constantes pequeñas en la práctica, pero se muestran explícitamente en las fórmulas de complejidad.

A.4 Cómo leer las fórmulas de complejidad

Cada algoritmo a continuación se describe con tres tipos de costo. Leerlos es más fácil si se tiene presente su significado en lenguaje llano:

- **Trabajo `W` (costo secuencial).** El número *total* de operaciones básicas que realiza el algoritmo, sin importar cuántos procesadores lo ejecuten. Es lo que se pagaría en un solo procesador. Nunca disminuye al añadir hardware, es el tamaño intrínseco del cómputo.

- **Tiempo paralelo `T_p` (tiempo de reloj con `p` procesadores).** Cuánto tarda el algoritmo cuando `p` trabajadores se ejecutan en paralelo. La forma recurrente es

$$T_p \approx \underbrace{\frac{W}{p_{eff}}}_{\text{trabajo repartido}} + \underbrace{\text{sobrecosto}}_{\text{coordinación, E/S, dependencias}} .$$

  El primer término es el trabajo dividido entre los trabajadores (aceleración perfecta). El segundo término es lo que impide que la aceleración sea perfecta: combinar resultados parciales (una *reducción*, que cuesta cerca del `log` del número de elementos), leer y escribir archivos, contabilidad del planificador y cualquier paso que deba esperar a uno anterior.

- **Span `T_∞` (el camino crítico).** El tiempo incluso con procesadores *ilimitados*, la longitud de la cadena más larga de pasos que deben ocurrir en orden. Un span pequeño significa que el algoritmo es "ancho" (altamente paralelo); un span grande significa que algo es inherentemente secuencial.

Dos notaciones se repiten y vale la pena leerlas literalmente:

- Un término **`+ log(...)`** es el costo de una **reducción** paralela, p. ej., sumar muchos números en uno solo por sumas pareadas repetidas toma cerca de `log` pasos. Es pequeño pero no nulo, razón por la cual `T_p` nunca alcanza exactamente `W/p`.
- **`min(p, B)`** aparece dondequiera que el trabajo se corte en `B` **tareas de bucket** independientes. Solo se pueden ejecutar a la vez tantos buckets como trabajadores se tengan y como buckets existan, así que el paralelismo utilizable es `min(p, B)`. Si `p > B`, los trabajadores sobrantes no tienen bucket que tomar y quedan ociosos, el origen de la regla $p_{eff} \leq B$ que se discute en la sección de HPC.

Así que, en una frase: **`W` dice cuán grande es el cómputo, `T_p` dice cuánto tiempo de reloj puede recuperar el clúster, y los términos de sobrecosto dicen por qué esa aceleración acaba deteniéndose.**

A.5 Algoritmo 1: pipeline completo de TDEW

```
Algorithm TDEW-PIPELINE
Input:
  TD monthly files for all IDs and dates
  TMIN monthly files for all IDs and dates
  Grid table containing the expected IDs
  Parameters: B buckets, h local DOY window, p processors

Output:
  Daily climatology per (ID, doy)
  Bucket-year merged training dataset
  Regression coefficients per (ID, doy)
  Optional repair coefficients for incomplete DOYs
  Optional TD predictions

1  climatology <- BUILD-CLIMATOLOGY(TD, TMIN)
2  bucketed_training <- BUILD-BUCKET-YEAR-DATASET(TD, TMIN, B)
3  bucketed_climatology <- SHARD-CLIMATOLOGY(climatology, B)
4  coefficients <- TRAIN-COEFFICIENTS(bucketed_training,
                                      bucketed_climatology,
```

```
                                    h,
                                    B)
5  incomplete_doys <- CHECK-COMPLETENESS(coefficients, Grid)
6  if incomplete_doys is not empty then
7      patch <- RERUN-FAILED-DOYS(bucketed_training,
                                  bucketed_climatology,
                                  incomplete_doys,
                                  h,
                                  B)
8      coefficients <- PATCH-COEFFICIENTS(coefficients,
                                          patch,
                                          incomplete_doys)
9      incomplete_doys <- CHECK-COMPLETENESS(coefficients, Grid)
10 end if
11 if forecasting is requested then
12     bucketed_future_tmin <- SHARD-FUTURE-TMIN(TMIN future, B)
13     predictions <- FORECAST-TD(coefficients,
                                  bucketed_climatology,
                                  bucketed_training,
                                  bucketed_future_tmin,
                                  H,
                                  B)
14 end if
15 return coefficients, predictions
```

Lectura del pipeline. El flujo es: primero aprender los valores normales estacionales de cada ubicación (**climatología**, línea 1); reorganizar los archivos mensuales crudos en archivos compactos por bucket para que las etapas posteriores lean de forma eficiente (**construir dataset bucket-año** y **fragmentar climatología**, líneas 2–3); ajustar las regresiones locales que son la salida científica (**entrenar coeficientes**, línea 4); luego un bucle de control de calidad (líneas 5–10) que verifica si cada día del año se ajustó con éxito para cada ubicación y, si no, reentrena *solo* los días faltantes y los integra; y finalmente un **pronóstico** opcional (líneas 11–14) que avanza el modelo ajustado hacia adelante sobre un horizonte futuro. Los pasos se ejecutan en secuencia, pero cada paso es internamente paralelo (sus bucles `pardo`). Los algoritmos posteriores expanden cada una de estas líneas.

A.6 Algoritmo 2: construir climatología

La climatología es la media histórica para cada `(ID, doy)`.

```
Algorithm BUILD-CLIMATOLOGY
Input:
  TD rows:   (ID, date, TD)
  TMIN rows: (ID, date, TMIN)

Output:
  climatology rows: (ID, doy, TD_clim, TMIN_clim)

1  for each climate variable V in {TD, TMIN} pardo do
2      for each monthly file m of V pardo do
3          read rows from file m
4          for each row r pardo do
```

```
5              r.doy <- day_of_year(r.date)
6              emit partial record (r.ID, r.doy, r.value)
7          end for
8      end for
9      for each key (ID, doy) pardo do
10         sum_V(ID, doy) <- parallel sum of values with this key
11         count_V(ID, doy) <- parallel count of values with this key
12         clim_V(ID, doy) <- sum_V(ID, doy) / count_V(ID, doy)
13     end for
14 end for
15 join TD_clim and TMIN_clim by (ID, doy)
16 return climatology
```

Complejidad:

$$W_{clim} = O(R)$$

$$T_p = O(R/p + \log R)$$

Espacio:

$$S_{clim} = O(ND)$$

Qué significan los costos. El trabajo `O(R)` indica que tocamos cada fila de cada variable climática exactamente una vez, no hay forma de calcular promedios sin mirar todos los datos, así que esto es tan barato como puede serlo. El tiempo paralelo `O(R/p + log R)` indica que el recorrido de filas se reparte perfectamente entre `p` trabajadores (`R/p`), más una pequeña cola `log R` para combinar las sumas parciales por clave en medias finales (la reducción). La salida es un número por par `(ID, doy)`, de ahí la memoria `O(ND)`, mucho menor que la entrada cruda, porque 40 años de valores diarios colapsan en una sola curva estacional por ubicación.

A.7 Algoritmo 3: construir el dataset de entrenamiento bucket-año

La fusión mensual de TD/TMIN se realiza una sola vez. Las filas se distribuyen luego a buckets de ID deterministas y se compactan por año.

```
Algorithm BUILD-BUCKET-YEAR-DATASET
Input:
  TD monthly files
  TMIN monthly files
  Number of buckets B

Output:
  bucket-year training files:
  bucket b, year y -> rows (ID, date, TD, TMIN, doy)

1  for each year y pardo do
2      for each month m in year y pardo do
3          TD_m <- read TD rows for month m
4          TMIN_m <- read TMIN rows for month m
5          M_m <- hash join TD_m and TMIN_m by (ID, date)
6          for each row r in M_m pardo do
7              r.doy <- day_of_year(r.date)
```

```
8              r.bucket <- r.ID mod B
9              emit r to temporary partition (bucket = r.bucket, year = y)
10         end for
11     end for
12     for each bucket b in 0..B-1 pardo do
13         read all temporary rows for (bucket b, year y)
14         sort rows by (ID, date)
15         write compact file for (bucket b, year y)
16     end for
17 end for
18 return bucket-year training dataset
```

Complejidad con joins por hash:

$$W_{bucket} = O(R)$$

Si se incluye el ordenamiento dentro de cada fragmento bucket-año:

$$W_{bucket-sort} = O\left(R + \sum_{b,y} n_{b,y} \log n_{b,y}\right)$$

Con buckets balanceados, $n_{b,y} \approx (N/B)D$, así que:

$$W_{bucket-sort} = O\left(R + BY \cdot \frac{ND}{B} \log\left(\frac{ND}{B}\right)\right) = O\left(R \log\left(\frac{ND}{B}\right)\right)$$

Tiempo paralelo ideal:

$$T_p = O\left(\frac{R}{p} + \log\left(\frac{ND}{B}\right)\right)$$

Memoria pico por compactación bucket-año:

$$S_{bucket-year} = O\left(\frac{ND}{B}\right)$$

La idea en una imagen. Un *bucket* es simplemente un grupo determinista de IDs: `bucket = ID mod B`. Pagamos la costosa fusión TD↔TMIN una sola vez, luego archivamos cada fila en su bucket (y año). Después, cualquier etapa que necesite los datos de un ID lee un único archivo pequeño, ya fusionado y ya ordenado, en lugar de reescanear todo el archivo histórico.

Lectura de las fórmulas. El trabajo `O(R)` indica de nuevo que pasamos por cada fila una vez. El factor `log(ND/B)` que aparece cuando se incluye el ordenamiento es el costo de ordenar las filas *dentro* de cada fragmento bucket-año; como cada fragmento contiene solo unas `ND/B` filas (no las `R` totales), ese log es de una cantidad pequeña, así que ordenar añade muy poco. El tiempo paralelo tiene el mismo reparto de trabajo `R/p` más esa pequeña reducción `log(ND/B)`. Crucialmente, la memoria pico es `O(ND/B)`, **no** `O(R)`: un trabajador solo mantiene un fragmento bucket-año a la vez, así que aumentar `B` *reduce* la memoria que cada trabajador necesita. Esa es la palanca que evita que 300 mil IDs × 40 años tengan que caber en la RAM de una sola máquina.

A.8 Algoritmo 4: fragmentar climatología por bucket

```
Algorithm SHARD-CLIMATOLOGY
Input:
  climatology rows (ID, doy, TD_clim, TMIN_clim)
  Number of buckets B

Output:
  bucketed climatology files

1  for each climatology row c pardo do
2      c.bucket <- c.ID mod B
3      emit c to bucket c.bucket
4  end for
5  for each bucket b in 0..B-1 pardo do
6      write climatology rows for bucket b
7  end for
8  return bucketed climatology
```

Complejidad:

$$W_{clim-shard} = O(ND)$$

$$T_p = O(ND/p + \log B)$$

Memoria pico con streaming:

$$S_{clim-shard} = O(ND/B)$$

Si se carga toda la climatología de una vez, la memoria pico se vuelve `O(ND)`.

Por qué existe este paso. La fragmentación no hace aritmética alguna, solo *mueve* cada fila de climatología al bucket al que pertenece su ID, de modo que la etapa de entrenamiento encuentre los valores normales de un ID en el mismo bucket que sus filas de entrenamiento y lea ambos en una sola pasada. El costo es por tanto una única pasada en streaming (trabajo `O(ND)`) más un término `log B` para enrutar filas a `B` destinos; el streaming mantiene en memoria solo el equivalente a un bucket (`O(ND/B)`) a la vez.

A.9 Algoritmo 4b: fragmentar TMIN futuro por bucket

El pronóstico necesita la `TMIN` futura exógena sobre el horizonte de predicción dispuesta por los mismos buckets que los coeficientes, de modo que cada tarea de pronóstico lea sus entradas una sola vez. Esto refleja el preprocesamiento bucket-año del Algoritmo 3, pero para una sola variable y solo sobre el horizonte de pronóstico.

Sea `R_fut = N * H` el número de filas de TMIN futura sobre el horizonte `H`.

```
Algorithm SHARD-FUTURE-TMIN-BY-BUCKET
Input:
  future TMIN monthly files over the forecast horizon
  Number of buckets B

Output:
  bucketed future-TMIN files: bucket b -> rows (ID, date, TMIN)
```

```
1  for each month m in the horizon pardo do
2      read future TMIN rows for month m
3      for each row r pardo do
4          r.bucket <- r.ID mod B
5          emit r to temporary partition (bucket = r.bucket)
6      end for
7  end for
8  for each bucket b in 0..B-1 pardo do
9      write compact future-TMIN file for bucket b
10 end for
11 return bucketed future TMIN
```

Complejidad:

$$W_{fut-shard} = O(R_{fut}) = O(NH)$$

$$T_p = O(NH/p + \log B)$$

Memoria pico con streaming:

$$S_{fut-shard} = O(NH/B)$$

En resumen. Este es el mismo paso de enrutamiento que el Algoritmo 4, pero aplicado a la `TMIN` futura que impulsa el pronóstico, y solo sobre los `H` días de pronóstico en lugar de toda la historia. Disponerla por los *mismos* buckets significa que cada tarea de pronóstico leerá después sus coeficientes, climatología, historia y `TMIN` futura, todo desde un mismo bucket, una sola vez.

A.10 Algoritmo 5: entrenar coeficientes de anomalía

Para cada `ID` y cada `doy` objetivo, ajustar una regresión ponderada sobre una vecindad circular de días del año.

Qué hace realmente un ajuste. Supongamos que queremos el modelo para la ubicación `i` en el día del año `d` (digamos el día 60). No usamos solo datos del día 60, un solo día a lo largo de 40 años son demasiados pocos puntos. En su lugar, reunimos cada observación cuyo día del año esté dentro de `h` días de `d` (la *ventana local*), envolviendo en el fin de año de modo que el día 365 sea vecino del día 1 (distancia *circular*). Los puntos más cercanos al día `d` reciben más influencia mediante un **peso tricube** que cae suavemente a cero en el borde de la ventana, así que el ajuste refleja "alrededor del día 60" sin dejar de estar centrado en el día 60. Sobre esa vecindad ponderada resolvemos una pequeña regresión por mínimos cuadrados ponderados de la anomalía de `TD` sobre la anomalía de `TMIN` y unas pocas anomalías rezagadas. El resultado, los coeficientes de regresión, se almacena para `(i, d)`. Repetimos esto para cada día del año y cada ubicación, razón por la cual el conteo total de ajustes es enorme aunque cada uno sea diminuto.

```
Algorithm TRAIN-COEFFICIENTS
Input:
  bucket-year training dataset
  bucketed climatology
  Local DOY half-window h
```

```
  Number of buckets B

Output:
  coefficients per (ID, doy)

1  for each bucket b in 0..B-1 pardo do
2      train_b <- read all bucket-year training rows for bucket b
3      clim_b <- read climatology rows for bucket b
4      for each ID i in bucket b pardo do
5          X_i <- rows of train_b for ID i
6          C_i <- rows of clim_b for ID i
7          sort X_i by date
8          for each row t in X_i pardo do
9              TD_anom(t) <- TD(t) - TD_clim(i, doy(t))
10             TMIN_anom(t) <- TMIN(t) - TMIN_clim(i, doy(t))
11         end for
12         for each row t in X_i pardo do
13             create lag features:
14               TD_anom_lag1(t), TD_anom_lag2(t), TMIN_anom_lag1(t)
15         end for
16         for each target day d in 1..366 pardo do
17             N_d <- rows where circular_distance(doy(t), d) <= h
18             remove rows in N_d with missing target or missing features
19             if size(N_d) >= minimum_sample_count then
20                 for each row t in N_d pardo do
21                     weight(t) <- tricube(circular_distance(doy(t), d) / h)
22                 end for
23                 beta <- weighted_least_squares(N_d)
24                 write coefficients for (ID i, doy d)
25             end if
26         end for
27     end for
28     write coefficient file for bucket b
29     write failure log for bucket b, if needed
30 end for
31 return coefficient dataset
```

Por regresión local:

$$W_{one-fit} = O(WF^2 + F^3)$$

Aquí `F` es constante, así que:

$$W_{one-fit} = O(W) = O(Yh)$$

De dónde viene `O(WF² + F³)`. Resolver un ajuste por mínimos cuadrados ponderados implica formar la matriz de ecuaciones normales `F × F` X^TWX a partir de `W` filas de muestra ponderadas (ese es el término `WF²`, cada una de las `F²` entradas de la matriz suma sobre `W` filas) y luego resolver el sistema lineal `F × F` resultante (el término `F³`, el costo de una pequeña factorización matricial). Como el número de características `F = 5` es una constante fija aquí, ambos términos colapsan a una constante por `W`, así que **un ajuste cuesta `O(W) = O(Yh)`**, proporcional a cuántas muestras de la vecindad ve.

Todo el entrenamiento de coeficientes:

$$W_{train} = O(NDW) = O(NDYh)$$

Eso es simplemente *un ajuste* (`O(Yh)`) por el *número de ajustes* (`N` ubicaciones × `D` días del año). Con `N ≈ 3×10⁵` y `D ≈ 366`, eso son aproximadamente `1.1×10⁸` ajustes independientes, este producto es el costo dominante del pipeline y el objetivo principal de los experimentos de escalamiento.

Span PRAM ideal si buckets, IDs, DOYs y las reducciones internas de cada ajuste son completamente paralelos:

$$T_\infty = O(\log W + F^3)$$

Lectura del span. Si tuviéramos procesadores ilimitados, los `1.1×10⁸` ajustes podrían ejecutarse todos a la vez, así que el único tiempo que queda es el camino crítico de *un* ajuste: una reducción `log W` para sumar la vecindad en las ecuaciones normales, más la resolución constante `F³`. En otras palabras, el algoritmo es extremadamente "ancho", casi nada es inherentemente secuencial dentro del entrenamiento. El término `log W` es profundidad de la versión PRAM fina, no trabajo adicional. La implementación CPU/Dask no realiza esa reducción entre trabajadores: distribuye buckets y ejecuta cada regresión completa dentro de una tarea.

Para la versión PRAM fina, la cota trabajo–span con `p` procesadores es:

$$T_p = O(NDYh/p + \log(Yh))$$

La ejecución práctica bucket-paralela tiene a lo sumo `B` tareas externas y no añade `log(Yh)` a su modelo medido, así que:

$$T_p = O\left(\frac{NDYh}{\min(p, B)} + \text{sobrecosto de E/S}\right)$$

Por qué `min(p, B)` y no `p`, y qué significa esto para una GPU. En la implementación, la unidad de planificación paralela es el **bucket**: un bucket es una tarea entregada a un trabajador. Así que el número de buckets corriendo *al mismo tiempo* no puede ser mayor que el número de trabajadores `p` ni mayor que el número de buckets `B`, de ahí `min(p, B)`. Esto es paralelismo a **nivel de bucket** (externo), y es lo único que `min(p, B)` acota.

Hay un segundo nivel de paralelismo independiente *dentro* de cada tarea de bucket: los `≈ (N/B)·D` ajustes individuales `5×5` de ese bucket son mutuamente independientes. Cómo se explota ese nivel interno es exactamente lo que difiere entre CPU y GPU:

- **CPU.** Cada trabajador es un proceso de un solo hilo (BLAS fijado a un hilo), así que recorre los ajustes de su bucket de uno en uno. El nivel interno *no* se paraleliza por trabajador; la aceleración proviene enteramente de ejecutar más buckets a la vez. Aquí `p` = número de procesos trabajadores (≈ núcleos), y se desean **muchos** buckets (`B ≥ 4p`) para que ningún núcleo quede ocioso.
- **GPU (A100).** Cada GPU es un trabajador, así que a nivel de bucket `p` = número de GPUs (a menudo solo 1–8) y `min(p, B)` es pequeño. Pero una GPU no procesa los ajustes de un bucket de uno en uno: ensambla las ecuaciones normales X^TWX para *todos* los `≈ (N/B)·D ≈ 10⁵` ajustes del bucket y los resuelve en una sola llamada **batched** (por lotes), repartiéndolos entre miles de núcleos CUDA. Ese ancho de lote interno, del orden de `10⁵` ajustes ofrecidos al dispositivo por oleada, despejados en milisegundos, es donde reside la velocidad de la GPU, y **`min(p, B)` no lo acota**. La implicación práctica invierte la regla de CPU: una GPU quiere

buckets *lo bastante grandes* como para que cada lote llene el dispositivo, así que no se empuja `B` muy alto en ejecuciones con GPU.

En ambos casos el trabajo *total* `O(NDYh)` es idéntico; el hardware solo cambia cómo se llenan los dos niveles de paralelismo. La sección "El hardware es simplemente más procesadores" más abajo lo precisa, y los experimentos de escalamiento miden qué hardware despeja más rápido los `1.1×10⁸` ajustes.

Memoria pico por tarea de bucket:

$$S_{train-bucket} = O\left(\frac{NT}{B} + \frac{ND}{B}\right)$$

El primer término es la serie temporal de entrenamiento del bucket. El segundo término es la climatología del bucket.

A.11 Algoritmo 6: verificar completitud

```
Algorithm CHECK-COMPLETENESS
Input:
  coefficients
  expected ID count from grid

Output:
  incomplete DOYs

1  expected_count <- number of IDs in grid
2  for each coefficient row c pardo do
3      emit key c.doy with value c.ID
4  end for
5  for each doy d in 1..366 pardo do
6      observed_count(d) <- distinct count of IDs emitted for d
7      if observed_count(d) < expected_count then
8          mark d as incomplete
9      end if
10 end for
11 return incomplete DOYs
```

Sea `C = ND` el número completo de filas de coeficientes.

$$W_{check} = O(C)$$

$$T_p = O(C/p + \log C)$$

Espacio:

$$S_{check} = O(C)$$

Qué verifica. Un ajuste `(ID, doy)` puede omitirse si su vecindad tuvo demasiadas pocas muestras válidas (línea 19 del entrenamiento). Este paso es una auditoría de cordura: para cada día del año, cuenta cuántas ubicaciones realmente obtuvieron un coeficiente y lo compara con cuántas dice la grilla que deberían existir. Cualquier día del año al que le falten algunas ubicaciones se marca para reparación. Es una única pasada barata sobre la tabla de coeficientes (`O(C)`), y su salida es diminuta, una breve lista de a lo sumo `D` días a rehacer.

A.12 Algoritmo 7: reparar DOYs fallidos

Solo se reentrenan los DOYs incompletos. Si `Q` es pequeño, esto es mucho más barato que reentrenar los 366 DOYs.

```
Algorithm RERUN-FAILED-DOYS
Input:
  bucket-year training dataset
  bucketed climatology
  incomplete DOYs Q
  Local DOY half-window h

Output:
  patch coefficient dataset

1  for each bucket b in 0..B-1 pardo do
2      train_b <- read all bucket-year training rows for bucket b
3      clim_b <- read climatology rows for bucket b
4      for each ID i in bucket b pardo do
5          recompute anomalies and lags for ID i
6          for each failed day d in Q pardo do
7              fit weighted local regression for (ID i, doy d)
8              write patch coefficients for (ID i, doy d)
9          end for
10     end for
11 end for
12 return patch coefficient dataset
```

Complejidad:

$$W_{repair} = O(NQYh)$$

$$T_p = O(NQYh/p + \log(Yh))$$

Por qué la reparación es barata. La reparación es estructuralmente idéntica al entrenamiento (mismos buckets, mismos ajustes ponderados) pero restringida a los `Q` días marcados en lugar de a todos los `D`. Así que su costo es el costo del entrenamiento con `D` reemplazado por `Q`: `O(NQYh)`. Cuando solo falló un puñado de días (`Q` $\ll$ `D`), esto es una fracción diminuta de un reentrenamiento completo, rehacemos los pocos días rotos, no los 366.

A.13 Algoritmo 8: parchar coeficientes

```
Algorithm PATCH-COEFFICIENTS
Input:
  base coefficient dataset
  patch coefficient dataset
  incomplete DOYs Q

Output:
  corrected coefficient dataset

1  for each bucket b in 0..B-1 pardo do
2      base_b <- read coefficient rows for bucket b
```

```
3      patch_b <- read patch rows for bucket b
4      base_keep <- rows in base_b whose doy is not in Q
5      corrected_b <- union(base_keep, patch_b)
6      remove duplicate (ID, doy), keeping the patch row
7      sort corrected_b by (ID, doy)
8      write corrected coefficients for bucket b
9  end for
10 return corrected coefficient dataset
```

Complejidad:

$$W_{patch} = O(ND + NQ)$$

$$T_p = O((ND + NQ)/p)$$

Memoria pico por bucket:

$$S_{patch-bucket} = O(ND/B + NQ/B)$$

Qué hace el parchado. En lugar de editar la gran tabla de coeficientes en su lugar, cada bucket se reconstruye a sí mismo: conserva las filas cuyo día del año *no* fue reparado, añade las filas recién reparadas y descarta cualquier `(ID, doy)` duplicado en favor de la reparación. Esto es pura contabilidad, una lectura, un filtro, una unión, un ordenamiento y una escritura por bucket, así que su costo es solo proporcional al tamaño de la tabla más el pequeño parche, sin nuevo ajuste de modelo.

A.14 Algoritmo 9: pronóstico opcional (bucketizado)

El pronóstico es paralelo entre buckets, y entre IDs dentro de un bucket, pero recursivo a través del tiempo para cada ID: el día `t` depende de la anomalía de TD predicha previamente, así que el bucle del horizonte no puede paralelizarse dentro de un mismo ID. Como en el entrenamiento, el trabajo se organiza por bucket de modo que cada fragmento de coeficientes, climatología, historia y TMIN futura se lea exactamente una vez. Los fragmentos de TMIN futura los produce el Algoritmo 4b; la historia (usada solo para sembrar los dos primeros rezagos) se lee del dataset de entrenamiento bucket-año.

```
Algorithm FORECAST-TD
Input:
  bucketed coefficients per (ID, doy)
  bucketed climatology per (ID, doy)
  bucketed history (to seed lags) and bucketed future TMIN
  forecast horizon H
  Number of buckets B

Output:
  predicted TD for each (ID, date)

1  for each bucket b in 0..B-1 pardo do
2      coeff_b <- read coefficient rows for bucket b
3      clim_b  <- read climatology rows for bucket b
4      hist_b  <- read history rows for bucket b
5      ftmin_b <- read future TMIN rows for bucket b
```

```
6       for each ID i in bucket b pardo do
7           initialize lagged TD/TMIN anomalies from hist_b(i)
8           for forecast day t from 1 to H do          // secuencial en t
9               d <- day_of_year(date_t)
10              beta <- coeff_b(i, d)
11              x <- current TMIN anomaly and lagged anomaly features
12              TD_anom_hat <- beta dot x
13              TD_hat <- clim_b(i, d) + TD_anom_hat
14              update lagged anomalies with TD_anom_hat and TMIN_anom(t)
15              write prediction (ID i, date_t, TD_hat)
16          end for
17      end for
18      write prediction file for bucket b
19  end for
20  return predictions
```

Complejidad:

$$W_{forecast} = O(NH)$$

Tiempo práctico bucket-paralelo (a lo sumo `B` tareas externas, recursivo sobre el horizonte dentro de cada ID):

$$T_p = O\left(\frac{NH}{\min(p, B)} + H\right)$$

Por qué el pronóstico tiene un `+ H` que el entrenamiento no tiene. Los ajustes de entrenamiento son mutuamente independientes, así que se paralelizan completamente. El pronóstico es diferente *dentro* de una ubicación: para predecir el día `t` alimentamos la anomalía *predicha* del día `t-1` (las características de rezago). El día 2 necesita el día 1, el día 3 necesita el día 2, y así sucesivamente, así que los `H` días de pronóstico para un solo ID deben calcularse *en orden*, no pueden repartirse entre trabajadores. Esa cadena inevitable de longitud `H` es el término `+ H`. Aún paralelizamos completamente entre los millones de *ubicaciones* (y entre buckets), así que el trabajo `O(NH)` se divide por `min(p, B)`; solo el avance temporal por ID sigue siendo secuencial. Como con el entrenamiento, la estructura de buckets acota el paralelismo efectivo en $p_{eff} \leq B$, y en una GPU aplica la misma imagen de dos niveles, las recursiones de muchos IDs avanzan un paso temporal juntas como una operación batched entre los núcleos CUDA.

Memoria pico por tarea de bucket:

$$S_{forecast-bucket} = O\left(\frac{NH}{B} + \frac{ND}{B}\right)$$

A.15 Tabla resumen de complejidad

La tabla recopila los tres costos para cada fase. Cada fila se lee así: *"así de grande es la fase (`W`), así se encoge su tiempo de reloj con `p` trabajadores (`T_p`), y cuánta memoria necesita un trabajador."* Las fases con `min(p, B)` en su `T_p` (entrenamiento, reparación, pronóstico) son las etapas de cómputo bucket-paralelas cuya aceleración se satura una vez que `p` alcanza `B`; las otras (solo `/p`) son pasadas limitadas por E/S que escalan con los trabajadores hasta que el sistema de archivos se convierte en el límite. La única entrada dominante es **entrenar coeficientes**, `O(NDYh)`.

Fase	Trabajo secuencial	Tiempo paralelo ideal con `p` procesadores	Objetivo de memoria pico
Construir climatología	`O(R)`	`O(R/p + log p)`	`O(ND)` salida
Construir dataset bucket-año	`O(R)` o `O(R log(ND/B))` con ordenamiento	`O(R log(ND/B)/p + log p)`	`O(ND/B)` por compactación
Fragmentar climatología	`O(ND)`	`O(ND/p + log B)`	`O(ND/B)` streaming
Fragmentar TMIN futura	`O(NH)`	`O(NH/p + log B)`	`O(NH/B)` streaming
Entrenar coeficientes	`O(NDYh)`	`O(NDYh/min(p,B))`	`O(NT/B + ND/B)` por bucket
Verificar completitud	`O(ND)`	`O(ND/p + log p)`	`O(ND)` o streaming por bucket
Reparar `Q` DOYs fallidos	`O(NQYh)`	`O(NQYh/min(p,B))`	igual que el bucket de entrenamiento
Parchar coeficientes	`O(ND + NQ)`	`O((ND + NQ)/p)`	`O(ND/B + NQ/B)` por bucket
Pronóstico horizonte `H`	`O(NH)`	`O(NH/min(p,B) + H)`	`O(NH/B)` por bucket de salida

La tabla muestra el tiempo de la implementación bucket-paralela. Para la variante PRAM fina, añadir `log(Yh)` al entrenamiento y a la reparación como span de la reducción interna de un ajuste; no es trabajo adicional ni un sobrecosto medido de Dask.

A.16 Complejidad global

El pipeline completo es una secuencia de fases paralelas, así que el trabajo total es la suma del trabajo de cada fase. Los términos dominantes son:

- preprocesamiento bucket-año,
- entrenamiento completo de coeficientes,
- reparación opcional de DOYs fallidos,
- pronóstico opcional.

Usando `R = NYD`, el trabajo secuencial total para el preprocesamiento más el entrenamiento completo sin reparación y sin pronóstico es:

$$W_{total} = O\left(R \log\left(\frac{ND}{B}\right) + NDYh\right)$$

Equivalentemente:

$$W_{total} = O\left(NYD \log\left(\frac{ND}{B}\right) + NDYh\right)$$

Si el costo de ordenamiento dentro de los archivos bucket-año se ignora o se trata como un detalle de implementación, la expresión más simple es:

$$W_{total} = O(NYD + NDYh) = O(NDYh)$$

porque `h` es el principal multiplicador de la regresión local.

Si se necesita reparación para `Q` DOYs fallidos, añadir:

$$W_{repair-total} = O(NQYh)$$

Así que el entrenamiento completo más la reparación es:

$$W_{total+repair} = O\left(NYD \log\left(\frac{ND}{B}\right) + NYh(D+Q)\right)$$

Si también se ejecuta el pronóstico para el horizonte `H`, añadir:

$$W_{forecast-total} = O(NH)$$

Por tanto, la expresión de extremo a extremo más completa es:

$$W_{end-to-end} = O\left(NYD \log\left(\frac{ND}{B}\right) + NYh(D+Q) + NH\right)$$

Para la ejecución paralela, las fases son secuencialmente dependientes, pero cada fase usa `pardo` internamente. Con `p` procesadores y `B` tareas de bucket, el tiempo paralelo teórico es:

$$T_p = O\left(\begin{array}{c} \frac{NYD\log(ND/B)}{p} + \frac{NDYh}{\min(p,B)} + \frac{NQYh}{\min(p,B)} \\ + \frac{NH}{\min(p,B)} + H + \log p + \tau_{fit} \end{array}\right)$$

Término por término, ese tiempo paralelo se lee como: construir el dataset bucket-año (`NYD log(ND/B) / p`), hacer el entrenamiento completo (`NDYh / min(p,B)`, el grande), reparar los días fallidos si los hay (`NQYh / min(p,B)`), ejecutar el pronóstico (`NH / min(p,B)`), pagar la inevitable recursión de pronóstico por ID (`+ H`), y pagar las consolidaciones binarias entre trabajadores (`+ log p`). El término `τ_fit` representa el camino crítico de un ajuste: `O(log(Yh))` en la PRAM fina u `O(Yh)` cuando el ajuste se ejecuta completo dentro de una tarea. Los términos de preprocesamiento se dividen por `p` (están limitados por el sistema de archivos, no por los buckets), mientras que las tres etapas de cómputo se dividen por `min(p, B)` (están limitadas por el número de buckets).

El término `+ H` permanece porque el pronóstico es recursivo en el tiempo para cada ID. Si no se ejecuta el pronóstico, eliminar `NH/min(p,B) + H`. Si no existen DOYs fallidos, eliminar el término `NQYh / min(p,B)`.

Para las ejecuciones medidas en este proyecto, el cuello de botella práctico es:

$$O(NDYh)$$

Ese es el costo de ajustar regresiones locales para muchos IDs y todos los días del año. La organización bucket-año mantiene acotada la E/S sin modificar este trabajo estadístico. Esta afirmación describe el régimen observado; no elimina asintóticamente el término de ordenamiento `NYD log(ND/B)` de la expresión completa.

A.17 Interpretación HPC

La complejidad de trabajo total no depende de `p` porque mide el número total de operaciones requeridas por el algoritmo. Los mismos joins, cálculos de anomalías, regresiones locales,

reparaciones y pronósticos deben computarse tanto si el trabajo corre en un procesador como si corre en un clúster universitario de cómputo de alto rendimiento.

En un sistema HPC, la medida relevante para el tiempo de ejecución es el tiempo de ejecución paralelo:

$$T_p = O\left(\frac{W}{p_{eff}} + \text{sobrecosto de E/S} + \text{sobrecosto de planificación} + \text{dependencias secuenciales}\right)$$

donde `p_eff` es el número efectivo de procesadores que el algoritmo puede usar. Puede ser menor que el número físico de núcleos disponibles debido al número de buckets, el ancho de banda de memoria, el ancho de banda del sistema de archivos, el sobrecosto del planificador y las dependencias entre etapas del pipeline.

Para el diseño actual de entrenamiento basado en buckets:

$$p_{eff} \leq B$$

Esto se debe a que la etapa principal de entrenamiento crea una gran tarea paralela por bucket. Si `B = 1024`, la etapa de entrenamiento puede exponer naturalmente hasta unas 1024 tareas de bucket paralelas. Si el HPC proporciona menos de 1024 ranuras de trabajador, el clúster puede usarse de forma eficiente. Si el HPC proporciona muchas más de 1024 ranuras de trabajador, los procesadores adicionales no reducirán mucho el tiempo de entrenamiento a menos que aumentemos el número de buckets o añadamos otro nivel de paralelismo dentro de cada bucket, por ejemplo dividiendo el trabajo por `ID` o por grupos de `doy`.

Una regla práctica para un despliegue HPC es:

$$B \geq 4p$$

donde `p` es el número de ranuras de trabajador planificadas para la ejecución. Esto le da al planificador suficientes tareas de bucket para balancear trabajadores lentos y rápidos. Sin embargo, `B` no debe hacerse arbitrariamente grande porque demasiados buckets pequeños crean muchos archivos pequeños y aumentan el sobrecosto de planificación.

Puntos de partida de ejemplo:

Ranuras de trabajador planificadas	Número de buckets sugerido
64	512 o 1024
128	1024 o 2048
256	2048 o 4096
512	4096 o 8192

Por tanto, para un HPC universitario, el proyecto debe evaluarse usando ambos:

- el trabajo total, que describe el tamaño del cómputo científico; y
- el tiempo de ejecución paralelo, que describe cuánto tiempo de reloj puede reducirse usando el clúster.

La aceleración esperada es fuerte mientras `p_eff` aumenta, pero acaba saturándose debido al ancho de banda de E/S, el sobrecosto del planificador, el número de buckets y la dependencia recursiva del pronóstico sobre el horizonte `H`.

A.18 Notas prácticas para esta máquina

Con unos 300 mil IDs, 40 años y 32 hilos lógicos de CPU, la mejor estrategia es mantener el número de buckets mucho mayor que el número de trabajadores. Por ejemplo, `B = 1024` da aproximadamente `N/B approx 293` IDs por bucket, lo que mantiene cada tarea de bucket lo bastante grande como para amortizar el sobrecosto pero lo bastante pequeña como para caber en memoria.

A.18.1 El hardware es simplemente más procesadores

La complejidad anterior es agnóstica al hardware: el trabajo total `W = O(NDYh)` es el mismo tanto si el pipeline corre en núcleos de CPU como en una GPU. El hardware cambia solo dos cosas: el número de procesadores paralelos `p` que expone, y el factor constante `c` de un solo ajuste local. El tiempo de ejecución paralelo es siempre

$$T_p = c \cdot \frac{W}{p_{eff}} + \text{sobrecosto} \ .$$

Los mínimos cuadrados ponderados por `(ID, doy)` son vergonzosamente paralelos: hay unos `N * D` (aproximadamente 1.1×10^8) ajustes independientes, cada uno una pequeña regresión ponderada de `F = 5` características sobre `W = O(Yh)` filas de vecindad. Esto se mapea igual de bien a:

- **CPU**: muchos procesos trabajadores de bucket de un solo hilo (uno por núcleo), donde la estructura de buckets acota $p_{eff} \leq B$; o
- **GPU**: un dispositivo que evalúa miles de estos ajustes diminutos concurrentemente una vez que se agrupan en lotes, ensamblando las ecuaciones normales ponderadas $F \times F$ $(X^\top W X)\beta = X^\top W y$ para muchos pares `(ID, doy)` y resolviéndolos en una sola llamada batched.

Dos niveles de paralelismo, y cuál llena una GPU. Vale la pena ser preciso sobre qué significa aquí "más procesadores", porque los dos hardwares añaden paralelismo en niveles diferentes:

- *Nivel externo (bucket)*, cuántos buckets corren a la vez. Esto es lo que `min(p, B)` acota. En CPU, `p` es el número de procesos trabajadores ($\approx$ núcleos); en GPU, `p` es el número de **GPUs** (un trabajador `dask-cuda` por dispositivo), a menudo solo 1–8.
- *Nivel interno (ajuste)*, cuántos de los $\approx (N/B)\, D \approx 10^5$ ajustes independientes de un bucket se evalúan juntos. En CPU esto es `1` (cada trabajador ajusta uno a la vez, BLAS fijado). En GPU esto es el **ancho de lote**: el dispositivo factoriza `10^5` sistemas 5×5 apilados a la vez. `min(p, B)` **no** acota este nivel.

Concretamente, una sola A100 (108 SMs, hasta ~2048 hilos residentes cada uno, así que $\approx 2.2 \times 10^5$ hilos en vuelo) recibe los $\approx 10^5$ ajustes de un bucket entero por oleada y los despeja en milisegundos. Así que una A100 da `p = 1` a nivel de bucket pero $\sim 10^5$ carriles de ajuste por oleada, la velocidad reside en el nivel interno. Esto también invierte la guía sobre el número de buckets: la CPU quiere **muchos** buckets ($B \geq 4p$) para mantener los núcleos ocupados, mientras que la GPU quiere buckets **lo bastante grandes** como para que cada lote llene el dispositivo, así que `B` no se empuja alto en ejecuciones con GPU.

Así que una GPU es, en términos PRAM, simplemente una fuente de un `p` *efectivo* mayor (con una constante `c` diferente): un `p` externo modesto multiplicado por un lote interno muy grande. Cuál de CPU o GPU produce el menor $c \cdot T_p$ para esta carga de trabajo es una pregunta

empírica, respondida por los experimentos de escalamiento de más abajo, no por el modelo de complejidad.

Las aceleraciones restantes, independientes del hardware, siguen viniendo de:

- usar archivos compactos bucket-año,
- mantener acotada la memoria del trabajador,
- paralelizar por bucket,
- y reparar solo los DOYs fallidos en lugar de repetir todo el entrenamiento.

A.19 Metodología experimental

Esta sección define cómo se evalúa la implementación en el HPC universitario. El objetivo es medir, como función del número de procesadores `p`, tres cantidades estándar de rendimiento paralelo, y hacerlo tanto en hardware de CPU como de GPU y en ambas versiones del dataset.

A.19.1 Cantidades medidas

Para un tamaño de problema fijo, sea `T(p)` el tiempo de reloj de una fase (o del pipeline de extremo a extremo) usando `p` procesadores. Reportamos:

$$\text{Tiempo de ejecución: } T(p) \qquad \text{Aceleración: } S(p) = \frac{T(1)}{T(p)} \qquad \text{Eficiencia: } E(p) = \frac{S(p)}{p}$$

En términos sencillos: la **aceleración** responde "¿cuántas veces más rápido lo hicieron `p` trabajadores?" y la **eficiencia** responde "¿qué fracción del tiempo de esos trabajadores fue realmente útil?". La aceleración ideal es `S(p) = p` (8 trabajadores → 8× más rápido) y la eficiencia ideal es `E(p) = 1` (100% de cada trabajador ocupado). La eficiencia real cae por debajo de 1 porque parte del trabajo es secuencial o los trabajadores se esperan entre sí o al disco. `S(p)` es idealmente lineal y `E(p)` es idealmente `1`. El análisis PRAM predice la desviación de lo ideal: como las etapas de entrenamiento y pronóstico son bucket-paralelas con $p_{eff} \leq B$, la eficiencia se mantiene alta mientras $p \leq B$ y luego se degrada; el ancho de banda de E/S, el sobrecosto del planificador y las dependencias secuenciales del pipeline fijan el techo práctico.

A.19.2 Definición de un "procesador" `p`

Para comparar CPU y GPU en un mismo eje, `p` cuenta ranuras de trabajador independientes:

Hardware	Un procesador (unidad `p`)	Cómo se varía `p`
CPU (SLURM)	un proceso trabajador de Dask de un solo hilo (un núcleo, BLAS fijado a 1 hilo)	`dask_jobqueue.SLURMCluster` escalado a `p` trabajadores
GPU (A100)	una GPU A100 (un trabajador `dask-cuda`)	`p` = número de A100s en la asignación

Dentro de una sola GPU, el solucionador batched de mínimos cuadrados ponderados usa todos los núcleos CUDA; ese paralelismo intra-dispositivo se mantiene fijo y se absorbe en la constante `c`, así que el eje `p` reportado es el número de GPUs. Esto mantiene `S(p)` y `E(p)` significativos a través del hardware: ambos responden "¿cuánto más rápido con `p` trabajadores independientes?".

A.19.3 Controles

Para hacer las mediciones reproducibles y justas:

- **La línea base `T(1)`** es un solo trabajador con `threads_per_worker = 1` y los pools de hilos de BLAS fijados a 1 (`OMP_NUM_THREADS=1`, etc.), como en `run_pipeline.sh`.
- **El número de buckets** se mantiene en $B \geq 4\,p_{max}$ durante todo el barrido para que la granularidad de buckets nunca sea el factor limitante antes de que `p` alcance su máximo.
- **Ensayos repetidos**: cada punto `(p, n)` se ejecuta `k` veces (p. ej., `k = 3`) y se reporta el tiempo de reloj mediano, para suprimir el jitter del sistema de archivos y del planificador.
- **Almacenamiento**: el scratch / spill del trabajador va a almacenamiento rápido local del nodo (`local_directory = $TMPDIR`); el parquet de entrada vive en el sistema de archivos compartido.
- **Caché caliente vs fría** se registra; las ejecuciones con caché fría (primer acceso) se reportan por separado porque las fases de preprocesamiento limitadas por E/S son sensibles a la caché.
- Solo se cronometra la fase bajo prueba (con `time.perf_counter` alrededor de la llamada al driver); las entradas de fases anteriores se precalculan una vez y se reutilizan.

Los dos estudios de escalamiento de abajo plantean dos preguntas diferentes. El **escalamiento fuerte** fija el problema y añade trabajadores, "¿puede el clúster hacer que *este* trabajo termine antes?". El **escalamiento débil** hace crecer el problema al ritmo de los trabajadores, "si duplicamos tanto los datos como las máquinas, ¿se mantiene plano el tiempo?". El escalamiento fuerte expone el techo $p_{eff} \leq B$; el escalamiento débil aísla el sobrecosto que crece con el número de trabajadores.

A.19.4 Escalamiento fuerte (problema fijo, más procesadores)

Mantener el tamaño del problema fijo en la grilla completa (o en un subconjunto representativo fijo de `N` IDs) y variar `p`:

$$p \in \{1, 2, 4, 8, 16, 32, ...\} \;\; \text{(CPU)} \qquad p \in \{1, 2, 4, 8\} \;\; \text{(GPUs A100)}$$

Para cada `p` registramos `T(p)` para la fase dominante de **entrenar-coeficientes** (y, por separado, las fases de construir-bucket y climatología limitadas por E/S, y opcionalmente el pronóstico). A partir de estas calculamos `S(p)` y `E(p)` y graficamos las tres contra `p`. La curva esperada sube casi linealmente mientras $p \leq B$ y la carga de trabajo es limitada por cómputo, luego se aplana, probando directamente la predicción $p_{eff} \leq B$.

A.19.5 Escalamiento débil (más procesadores, problema proporcionalmente mayor)

Mantener constante el trabajo por procesador escalando el número de IDs entrenados con `p`:

$$n(p) = n_0 \cdot p$$

donde `n` es el número de IDs entrenados. El subconjunto de IDs se selecciona de forma determinista mediante el parámetro `bucket_ids` existente (cada bucket contiene $\approx N/B$ IDs, así que elegir $\lceil 4p \rceil$ buckets da un `n(p)` controlado). La eficiencia de escalamiento débil es

$$E_{weak}(p) = \frac{T(1, n_0)}{T(p, n_0 p)}$$

que es idealmente `1`. Como los ajustes por `(ID, doy)` son independientes, se espera que la parte de cómputo del entrenamiento escale débilmente bien; las desviaciones aíslan el sobrecosto de E/S y de planificación que crece con el número de trabajadores.

A.19.6 Datasets: PISCOt v1.1 vs v1.2

El protocolo completo de escalamiento fuerte y débil se ejecuta en ambas versiones del dataset. Las dos versiones se suministran al mismo pipeline mediante los parámetros de ruta de entrada /

carpeta de variables (`--base`, `--td-var`, `--tmin-var`) escribiendo a raíces de resultados separadas, así que no difiere ningún código entre ejecuciones. Comparamos:

- **Rendimiento**: curvas `T(p)`, `S(p)`, `E(p)` para v1.1 vs v1.2 (el trabajo `W` difiere solo si `N`, `Y` o la densidad de datos difieren entre versiones).
- **Ciencia**: distribuciones de los coeficientes ajustados y de la habilidad predictiva, para evaluar cómo la versión del dataset afecta el modelo estimado, independientemente del tiempo de ejecución.

A.19.7 Configuraciones de hardware

Configuración	Clúster	`p` barrido
CPU multinodo	`dask_jobqueue.SLURMCluster` (un proceso por núcleo, $B \geq 4p_{max}$)	procesos trabajadores
GPU una/varias A100	`dask-cuda` (`LocalCUDACluster` o SLURM `--gres=gpu:a100:p`)	número de A100s

A.19.8 Reportes

Para cada combinación (dataset × hardware) producimos: (1) una tabla de `T(p)`, `S(p)`, `E(p)` para el barrido de escalamiento fuerte; (2) una tabla de $E_{weak}(p)$; y (3) tres gráficas, tiempo de ejecución vs `p`, aceleración vs `p` (con la referencia ideal `S(p)=p`) y eficiencia vs `p`. Estas cuantifican tanto cómo escala la implementación como qué hardware minimiza el tiempo de reloj para esta carga de trabajo.